This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

IEEE ComSoc IEEE Communications Society | IEEE COMPUTER SOCIETY | IEEE Signal Processing Society | IEEE VTS Connecting the Mobile World | IEEE Transactions on Machine Learning in Communications and Networking

Received 17 June, 2023; revised 23 November, 2023; accepted 10 January, 2024; Date of publication XX Month, XXXX; date of current version XX Month, XXXX.

Digital Object Identifier 10.1109/TMLCN.

Learning Resource Allocation Policy: Vertex-GNN or Edge-GNN?

YAO PENG[1], JIA GUO[1] (Graduate Student Member, IEEE), AND CHENYANG YANG[1] (Senior Member, IEEE)

[1]School of Electronics and Information Engineering, Beihang University, Beijing, China

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62271024 and the National Key R&D Program of China under Grant 2022YFB2902002.

ABSTRACT Graph neural networks (GNNs) update the hidden representations of vertices (called Vertex-GNNs) or hidden representations of edges (called Edge-GNNs) by processing and pooling the information of neighboring vertices and edges and combining to exploit topology information. When learning resource allocation policies, GNNs cannot perform well if their expressive power is weak, i.e., if they cannot differentiate all input features such as channel matrices. In this paper, we analyze the expressive power of the Vertex-GNNs and Edge-GNNs for learning three representative wireless policies: link scheduling, power control, and precoding policies. We find that the expressive power of the GNNs depends on the linearity and output dimensions of the processing and combination functions. When linear processors are used, the Vertex-GNNs cannot differentiate all channel matrices due to the loss of channel information, while the Edge-GNNs can. When learning the precoding policy, even the Vertex-GNNs with non-linear processors may not be with strong expressive ability due to the dimension compression. We proceed to provide necessary conditions for the GNNs to well learn the precoding policy. Simulation results validate the analyses and show that the Edge-GNNs can achieve the same performance as the Vertex-GNNs with much lower training and inference time.

INDEX TERMS *Vertex-GNN, Edge-GNN, resource allocation, precoding, expressive power.*

I. INTRODUCTION

OPTIMIZING resource allocation such as link scheduling, power control, and precoding is important for improving the spectral efficiency of wireless communications. Various numerical algorithms have been proposed to solve these problems, such as the weighted minimal mean-square error (WMMSE) and the fractional programming algorithms [1], [2], which are however with high computational complexity. To facilitate real-time implementation, fully-connected neural networks (FNNs) have been introduced to learn resource allocation policies, which are the mappings from environmental parameters (e.g., channels) to the optimized variables [3]. While significant research efforts have been devoted to intelligent communications, most existing works optimize resource allocation with FNNs or convolutional neural networks (CNNs). These DNNs are with high training complexity, not scalable, and not generalizable to problem size (say, the number of users). This hinders their practical use in dynamic wireless environments.

Encouraged by the potential for achieving good performance, reducing sample complexity and space complexity, as well as supporting scalability and size generalizability, graph neural networks (GNNs) have been introduced for learning resource allocation policies [4]–[9].

These benefits of GNNs originate from exploiting topology information of graphs and permutation properties of wireless policies. To embed the topology information, the hidden representations in each layer of a GNN are updated by first aggregating the information of neighboring vertices and edges and then combining with the hidden representations in the previous layer, where the aggregation operation consists of processing and pooling. A policy can be learned either by a Vertex-GNN that updates the hidden representations of vertices or by an Edge-GNN that updates the hidden representations of edges, no matter if the optimization variables are defined on vertices or edges. To harness the permutation property, parameter-sharing should be introduced for each layer of a GNN. It has been shown

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

in [9] that a GNN designed for learning a wireless policy will not perform well if the permutation property of the functions learnable by the GNN is mismatched with the policy.

However, even after satisfying the permutation property of a policy, a GNN may still not perform well due to the insufficient expressive power for learning the policy. The expressive power of a GNN is weak if the GNN cannot distinguish some pairs of graphs [10]. When learning a wireless policy, the graphs (i.e., the samples) that a GNN learns over are with different features. If a GNN maps different inputs (i.e., features) into the same output (called action), then the policy learned by the GNN may not achieve fairly good performance. Nonetheless, such a cause of the performance degradation has never been noticed until now.

A. RELATED WORKS

1) Vertex-GNNs and Edge-GNNs

GNNs can either update the hidden representations of vertices (i.e., Vertex-GNNs) or edges (i.e., Edge-GNNs). In the literature of machine learning, Vertex-GNNs were proposed for "vertex-level tasks" (say, vertex and graph classification) whose actions are defined on vertices [11]. For "edge-level tasks" (say, edge classification and link prediction) whose actions are defined on edges, both Vertex-GNNs and Edge-GNNs have been designed. In early works as summarized in [11], edge-level tasks were learned by Vertex-GNNs with a read-out layer, which was used to map the representations of vertices to the actions on edges. In [12], [13], the edges of the original graph were transformed into vertices of a line-graph or hyper-graph. Then, the edge-level task on the original graph is equivalent to the vertex-level task on the line-graph or hyper-graph, which can be learned by Vertex-GNNs. In [14], an Edge-GNN was designed for learning an edge-level task.

In the literature of intelligent communications, GNNs have been designed to learn the link scheduling [4], [5], [15], [16], power control [6], [17] and power allocation [9] policies in device-to-device (D2D) communications and interference networks, the precoding policies in multi-user multi-input-multi-output (MIMO) system [18]–[21], as well as the access point selection policy in cell-free MIMO systems [22]. Except [19], [21] where the optimized precoding matrices were defined as the actions on edges and hence Edge-GNNs were designed, the actions of all these works were defined on vertices and thereby Vertex-GNNs were designed.

2) Structure of GNNs

When using GNNs for a learning task, graphs need to be constructed and the structures of GNNs need to be designed or selected. For a resource allocation task, more than one graph can be constructed, say homogeneous graph or heterogeneous graph, and the action can be defined either on vertex or on edge. The structure of a GNN is captured by its update equation, which can either be vertex-representation update or edge-representation update, with a variety of choices for the processing, pooling, and combination functions. The commonly used pooling functions are sum, mean, and max functions, and the processing and combination functions can be either linear or nonlinear (say using FNNs). The three functions used in the GNNs for wireless policies are provided in Table 1, which were designed empirically without explaining the rationality.

TABLE 1. Processing, pooling and combination functions of GNNs for resource allocation

Processing Function	Pooling Function	Combination Function	Literature
Linear function	sum	FNN without hidden layer	[5], [9], [19], [21]
FNN	max	FNN without hidden layer	[22]
FNN	max,mean	FNN	[16]
FNN	max	FNN	[6], [17], [18]
FNN	sum	FNN	[20]
FNN	mean	FNN	[18]
CNN	mean	FNN	[15]

GNNs can be designed to satisfy permutation properties, which widely exist in wireless policies [21], [23]. According to the problem, a policy can be with different properties such as one-dimension (1D)-permutation equivariance (PE), two-dimension (2D)-PE, and joint-PE [9] property. A GNN learning over a homogenous graph can automatically learn the policies with 1D-PE property if the read-out layer (if needed) also satisfies the 1D-PE property, since the permutation of the vertices in the graph does not affect the output of the GNN. If a GNN is designed for learning a policy with 2D-PE, joint-PE, or more complicated PE property, then the parameter-sharing in the update equation of the GNN should be judiciously designed, as detailed in [21]. In [4], [6], [9], [19], [21], the PE properties of their considered wireless tasks were analyzed, and the GNNs were designed to satisfy the same properties. A GNN with matched PE property to a policy is possible to be sample efficient, scalable, and size generalizable, but the GNN may still not perform well due to insufficient expressive power to the policy.

3) Expressive Power of GNNs

The parameter-sharing in GNNs restricts their expressive power [24]. In the literature of machine learning, the expressive power of Vertex-GNNs has been investigated for vertex classification, link prediction, and graph classification tasks, where the samples for training and testing a GNN are graphs with different topologies [10], [24]–[26]. The expressive power is characterized by the capability to distinguish non-isomorphic graphs.[1]

[1]According to the definition in [24], we know that two graphs are isomorphic if a graph can be obtained by reordering the vertices in another graph, otherwise they are non-isomorphic.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

The 1-dimensional Weisfeiler-Lehman (1-WL) test is a widely used algorithm to distinguish non-isomorphic graphs, which consists of an injective aggregation process. Both the 1-WL test and Vertex-GNNs iteratively update the representation of a vertex by aggregating the representations of its neighboring vertices. Inspired by such a finding, the expressive power of Vertex-GNNs was characterized by whether their aggregating functions were injective in [10]. To build a GNN that is as powerful as the 1-WL test, a graph isomorphic network (GIN) was developed in [10], which is a Vertex-GNN whose processing and combination functions are FNNs and the pooling function is sum function. Under the assumption that the features of vertices were from a countable set, e.g., all vertices are with identical feature, the aggregating functions of the GIN were proved to be injective. When replacing the processing functions in the GIN with linear functions or replacing the pooling functions with mean or max function, the aggregating functions were proved to be non-injective, hence the resultant GNN is with weaker expressive power than the GIN. It was empirically shown that the less powerful Vertex-GNNs perform worse than the GIN on a number of graph classification tasks.

In [24]–[26], the expressive power of Vertex-GNNs and the techniques to improve their expressive power were reviewed. According to the analyses in [10], a GNN as powerful as the 1-WL test is with the strongest expressive power among all Vertex-GNNs. Since the 1-WL test cannot distinguish some non-isomorphic graphs, e.g., k-regular graphs with the same size and same vertex features, the Vertex-GNNs also cannot distinguish them. To design Vertex-GNNs with stronger expressive power for these graphs, one can provide each vertex a unique feature to make vertices more distinguishable [24], [26] or use a high-order GNN that updates the representation of k-tuple of vertices to maintain more structural information of the graph [24]–[26]. However, these techniques incur computational costs.

Due to the considered tasks in [10], [24]–[26] and the references therein, these works did not consider the features of edges. When learning wireless policies, the edges of the constructed graphs may have features, and the graphs with the same topology but with different features usually corresponds to different actions. The 1-WL test was proposed to distinguish non-isomorphic graphs without edge features that are different from the graphs for wireless problems, hence the analyses for the expressive power of GNNs in [10], [24]–[26] are not applicable to most wireless problems. The expressive power of a GNN for learning wireless policies is captured by its capability to distinguish input features (e.g., channel matrices), which has never been investigated so far.

B. MOTIVATION AND MAJOR CONTRIBUTIONS

In this paper, we strive to analyze the impact of the structure of a GNN for optimizing resource allocation or precoding on its expressive power, aiming to provide useful insight into the design of GNNs for learning wireless policies.

We take the link scheduling and power control problems in D2D communications and the precoding problem in multi-user MIMO system as examples, each representing a class of policies. The link scheduling and power control policies are the mappings from the channel matrices to the optimized vectors in a lower dimensional space, where the optimization variables are respectively discrete and continuous scalars. The precoding policy is the mapping from the channel matrix to the optimized precoding matrix, where the optimization variables are vectors. To demonstrate that the graph constructed for a wireless policy is not unique and the policy can be learned by both Vertex-GNN and Edge-GNN, we consider different graphs and GNNs for each policy.

To the best of our knowledge, this is the first attempt to analyze the expressive power of the GNNs for learning wireless policies. The major contributions are summarized as follows.

- We find that Vertex-GNNs with linear processing functions cannot differentiate specific channel matrices due to the loss of channel information after aggregation, which leads to poor learning performance. Their expressive power can be improved by introducing non-linear processers. By contrast, the update equations of Edge-GNNs with linear processors do not incur the information loss.
- We find that the output dimensions of the processing and combination functions also restrict the expressive power of the Vertex-GNN for learning the precoding policy, in addition to the linear processors. We provide a lower bound on the dimensions for a Vertex-GNN or Edge-GNN without the dimension compression.
- We validate the analyses and compare the performance of Vertex-GNNs and Edge-GNNs for learning the three policies via simulations. Our results show that both the training time and inference time of the Edge-GNNs with linear processors are much lower than the Vertex-GNNs with FNN-processors to achieve the same performance.

The rest of this paper is organized as follows. In section II, we introduce three resource allocation policies. In section III, we present Vertex-GNNs and Edge-GNNs to learn the policies over the directed homogeneous or undirected heterogeneous graph, and analyze the expressive power of the GNNs with linear processors. In section IV, we analyze the impacts of the linearity and output dimensions of processing and combination functions on the expressive power of GNNs. In sections V and VI, we provide simulation results and conclusions.

Notations: $(\cdot)^{\mathsf{T}}$ and $(\cdot)^{\mathsf{H}}$ denote transpose and Hermitian transpose, respectively. $|\cdot|$ denotes the absolute value of a real number or the magnitude of a complex number. $\mathsf{Tr}(\cdot)$ denotes the trace of a matrix. $\mathbf{X} = [x_{ij}]_{m\times n}$ denotes a matrix with m rows and n columns where x_{ij} is the element in the ith row and the jth column. $\|\mathbf{X}\| \triangleq \sum_{i=1}^{m}\sum_{j=1}^{n} x_{ij}$,

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

and $|\mathbf{X}|_{max} \triangleq \max_{i=1}^{m} \max_{j=1}^{n} |x_{ij}|$. $\mathbf{\Pi}$, $\mathbf{\Pi_1}$ and $\mathbf{\Pi_2}$ denote permutation matrices. $\mathbb{R}$, $\mathbb{C}$, and $\mathbb{I}$ denote the sets of real, complex, and integer numbers, respectively. $\mathbb{R}^n$ denotes n-dimensional vector space.

II. RESOURCE ALLOCATION PROBLEMS AND POLICIES

In this section, we present three representative resource allocation problems.

1) Link Scheduling

Consider a D2D communication system with K pairs of transceivers. Every transmitter sends data to a receiver, and all the transmissions share the same spectrum. Hence, there exist interference among the transceiver pairs, as illustrated in Fig. 1(a). To coordinate the interference, not all the D2D links are activated. A link scheduling problem that maximizes the sum rate of active links is [2], [5], [15],

$$\max_{x_1,\cdots,x_K} \sum_{k=1}^{K} \log_2\left(1+\frac{x_k p_k \alpha_{kk}}{\sum_{j=1,j\neq k}^{K} x_j p_j \alpha_{jk}+\sigma_0^2}\right) \quad \text{s.t. } x_k \in \{0,1\},\ k=1,\cdots,K, \tag{1}$$

where x_k is the active state of the kth D2D link, $x_k = 1$ when the kth D2D link is active, $x_k = 0$ otherwise, α_{jk} is the composite large- and small-scale channel gain from the jth transmitter to the kth receiver, p_k is the power of the kth transmitter, and σ_0^2 is noise power.

The link scheduling policy is denoted as $\mathbf{x}^* = F_{\mathsf{ls}}(\boldsymbol{\alpha})$, where $\mathbf{x}^* = [x_1^*, \cdots, x_K^*]^{\mathsf{T}}$ is the optimized solution of the problem in (1) for a given channel matrix $\boldsymbol{\alpha} = [\alpha_{ij}]_{K\times K}$, and $F_{\mathsf{ls}}(\cdot)$ is a function that maps $\boldsymbol{\alpha} \in \mathbb{R}^{K\times K}$ into $\mathbf{x}^* \in \mathbb{I}^{K\times 1}$. This policy is joint-PE to $\boldsymbol{\alpha}$ [9], i.e., $\mathbf{\Pi}^{\mathsf{T}}\mathbf{x}^* = F_{\mathsf{ls}}(\mathbf{\Pi}^{\mathsf{T}}\boldsymbol{\alpha}\mathbf{\Pi})$.

2) Power Control

The interference in the D2D system can also be coordinated by adjusting the transmit power of every transmitter. The power control problem that maximizes the sum rate under power constraint is [3], [6], [17],

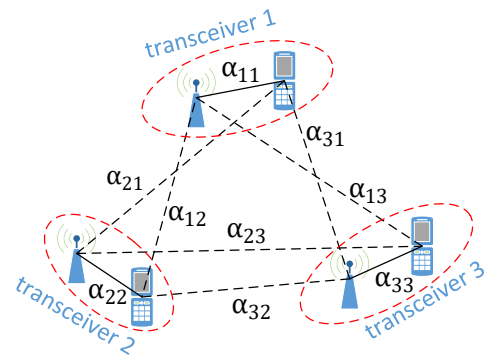
$$\max_{p_1,\cdots,p_K} \sum_{k=1}^{K} \log_2\left(1+\frac{p_k \alpha_{kk}}{\sum_{j=1,j\neq k}^{K} p_j \alpha_{jk}+\sigma_0^2}\right) \quad \text{s.t. } 0 \le p_k \le P_{max},\ k=1,\cdots,K, \tag{2}$$

where P_{max} is the maximal transmit power.

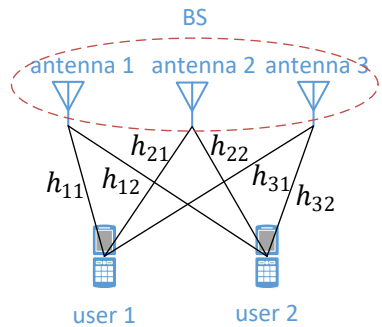
The power control policy is denoted as $\mathbf{p}^* = F_{\mathsf{pc}}(\boldsymbol{\alpha})$, where $\mathbf{p}^* = [p_1^*, \cdots, p_K^*]^{\mathsf{T}}$ is the optimized solution of the problem in (2) for a given channel matrix $\boldsymbol{\alpha}$, and $F_{\mathsf{pc}}(\cdot)$ is a function that maps $\boldsymbol{\alpha} \in \mathbb{R}^{K\times K}$ into $\mathbf{p}^* \in \mathbb{R}^{K\times 1}$. This policy is also joint-PE to $\boldsymbol{\alpha}$ [6], i.e., $\mathbf{\Pi}^{\mathsf{T}}\mathbf{p}^* = F_{\mathsf{pc}}(\mathbf{\Pi}^{\mathsf{T}}\boldsymbol{\alpha}\mathbf{\Pi})$.

3) Precoding

Consider a multi-user multi-antenna system, where a base station (BS) equipped with N antennas transmits to K users each with a single antenna, as shown in Fig. 1(b). The precoding problem that maximizes the sum rate of users



(a) D2D communication system



(b) Multi-user MIMO system

FIGURE 1. The considered two systems. (a): $K = 3$, solid and dashed lines represent the communication and interference links, respectively. (b): $N = 3$ and $K = 2$.

subject to power constraint is [19], [20]

$$\max_{\mathbf{v}_1,\cdots,\mathbf{v}_K} \sum_{k=1}^{K} \log_2\left(1+\frac{|\mathbf{h}_k^{\mathsf{H}}\mathbf{v}_k|^2}{\sum_{j=1,j\neq k}^{K} |\mathbf{h}_k^{\mathsf{H}}\mathbf{v}_j|^2+\sigma_0^2}\right) \quad \text{s.t. } \mathrm{Tr}(\mathbf{V}\mathbf{V}^{\mathsf{H}}) \le P_{max}, \tag{3}$$

where $\mathbf{V} = [\mathbf{v}_1, \cdots, \mathbf{v}_K] = [v_{nk}]_{N\times K}$, $\mathbf{v}_k \in \mathbb{C}^{N\times 1}$ is the precoding vector for the kth user, and $\mathbf{h}_k \in \mathbb{C}^{N\times 1}$ is the channel vector from the BS to the kth user.

Denote the precoding policy as $\mathbf{V}^* = F_{\mathsf{p}}(\mathbf{H})$, where $\mathbf{V}^*$ is the optimized solution of the problem in (3) for a given channel matrix $\mathbf{H} = [\mathbf{h}_1, \cdots, \mathbf{h}_K] = [h_{nk}]_{N\times K}$, and $F_{\mathsf{p}}(\cdot)$ is a function that maps $\mathbf{H} \in \mathbb{C}^{N\times K}$ into $\mathbf{V}^* \in \mathbb{C}^{N\times K}$. The precoding policy is 2D-PE to $\mathbf{H}$ [19], i.e., $\mathbf{\Pi}_1^{\mathsf{T}}\mathbf{V}^*\mathbf{\Pi}_2 = F_{\mathsf{p}}(\mathbf{\Pi}_1^{\mathsf{T}}\mathbf{H}\mathbf{\Pi}_2)$.

The link scheduling problem is a combinatorial optimization problem, which can be solved by using iterative algorithms [2] or exhaustive searching. Both the problems in (2) and (3) are non-convex, which can be solved by numerical algorithms such as the WMMSE algorithm [1].

III. GNNS FOR LEARNING THE POLICES

In this section, we introduce several GNNs to learn the three policies. Since constructing appropriate graphical models is the premise of applying GNNs, we first introduce graphs for each policy. Then, we introduce Vertex-GNN and

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

Edge-GNN. Finally, we analyze the expressive power of the GNNs with simple pooling, processing, and combination functions.

A. HOMOGENEOUS AND HETEROGENEOUS GRAPHS

A graph consists of vertices and edges. Each vertex or edge may be associated with features and actions. Learning a resource allocation policy over a graph is to learn the actions defined on vertices or edges based on their features. The inputs and outputs of a GNN are the features and actions of a graph, respectively.

A graph may consist of vertices and edges that belong to different types. If a graph consists of vertices or edges with more than one type, then it is a heterogeneous graph. Otherwise, it is a homogeneous graph, which can be regarded as a special heterogeneous graph.

More than one graph can be constructed for a resource allocation problem.

1) Link Scheduling/Power Control

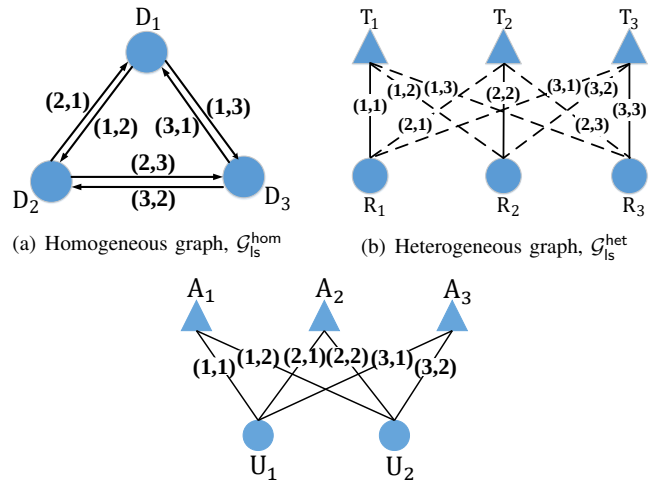
For learning the link scheduling and power control policies, the topologies and features of the graphs and the structures of the GNNs are the same, and only the actions of the GNNs and the loss functions for training the GNNs are different. Hence, we focus on the GNNs for learning the link scheduling policy in the sequel.

In [5], [15], a directed homogeneous graph $\mathcal{G}_{\mathsf{ls}}^{\mathsf{hom}}$, as shown in Fig. 2(a), was constructed for learning the link scheduling policy. In $\mathcal{G}_{\mathsf{ls}}^{\mathsf{hom}}$, each D2D pair is a vertex, and the interference links among the D2D pairs are directed edges. Denote the ith vertex as D_i, and the edge from D_i to D_j as edge (i,j). The feature of vertex D_i is α_{ii}, and the feature of edge (i,j) is α_{ij}. The features of all vertices and edges can be represented as $\boldsymbol{\alpha}$. The action of vertex D_i is the active state of the ith link x_i. In [6], the graph constructed for optimizing power control only has one difference from $\mathcal{G}_{\mathsf{ls}}^{\mathsf{hom}}$: the action of vertex D_i is p_i.

We can also construct an undirected heterogeneous graph $\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$ as shown in Fig. 2(b) for learning the link scheduling policy. In $\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$, there are two types of vertices and two types of edges. Each transmitter and each receiver are respectively defined as a transmitter vertex and a receiver vertex (respectively called tx vertex and rx vertex for short), and the link between them is an undirected edge. Denote the ith tx vertex and the ith rx vertex as T_i and R_i, respectively, and the edge between T_i and R_j as edge (i,j). Edge (i,i) is referred to as signal edge, and edge (i,j) $(i \neq j)$ is referred to as interference edge (respectively called sig edge and int edge for short). The vertices have no features. The feature of edge (i,j) is α_{ij}, and the features of all the edges can be represented as $\boldsymbol{\alpha}$. The active state x_i can either be defined as the action of vertex T_i or the action of sig edge (i,i).

2) Precoding

In [19], the precoding policy was learned over a heterogeneous graph $\mathcal{G}_{\mathsf{p}}^{\mathsf{het}}$ as shown in Fig. 2(c). In $\mathcal{G}_{\mathsf{p}}^{\mathsf{het}}$, there are two types of vertices and one type of edges. Each antenna at the BS and each user are respectively antenna vertex and user vertex, and the link between them is an undirected edge. Denote the ith antenna vertex and the jth user vertex as A_i and U_j, respectively, and the edge between A_i and U_j as edge (i,j). The vertices have no features. The feature of edge (i,j) is h_{ij}, and the features of all the edges can be represented as $\mathbf{H}$. The action v_{ij} is defined on edge (i,j).



(a) Homogeneous graph, $\mathcal{G}_{\mathsf{ls}}^{\mathsf{hom}}$ (b) Heterogeneous graph, $\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$

(c) Heterogeneous graph, $\mathcal{G}_{\mathsf{p}}^{\mathsf{het}}$

FIGURE 2. The topologies of the graphs. (a) and (b): link scheduling/power control, $K=3$. (c): precoding, $N=3$, $K=2$. The circles and triangles represent vertices, solid and dashed lines represent the two types of edges.

One can also construct a homogeneous graph for learning the precoding policy in a similar way as in [18]. In particular, each user (say the kth user) and all antennas at the BS are defined as a vertex, $\mathbf{h}_k$ and $\mathbf{v}_k$ are the feature and action of the vertex, respectively. Every two vertices are connected by an edge, which has no feature and action. However, a GNN learning over such a graph is only permutation equivariant to users, losing the property of permutation equivariance to antennas and hence incurring higher training complexity.

B. VERTEX-GNN AND EDGE-GNN

GNNs can be classified into Vertex-GNNs and Edge-GNNs, which respectively update the hidden representations of vertices and edges. Each class of GNNs can learn over either homogeneous or heterogeneous graphs. The directed homogeneous graph can be transformed into an undirected heterogeneous graph by regarding the edges of different directions as two types of undirected edges and the two vertices connected by a directed edge as two types of vertices.[2] Hence, we focus on undirected heterogeneous graph in the following (called heterogeneous graph for short).

[2]The basic idea is to transform the direction information in a directed homogeneous graph into the type information in an undirected heterogeneous graph, which is applicable to any problem.

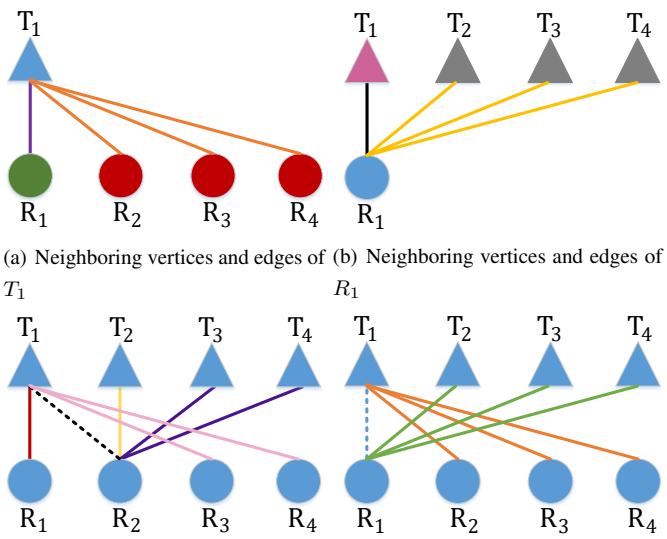
This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

For conciseness, we take the GNNs for learning the link scheduling policy over the heterogeneous graph $\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$ in Fig. 2(b) as an example. We discuss the GNN for learning the link scheduling policy over the converted heterogeneous graph from the directed homogeneous graph and the GNN for learning the precoding policy in remarks.

1) Vertex-GNN

In Vertex-GNN, the hidden representation of each vertex is updated in each layer, by first aggregating information from its neighboring vertices and edges, and then combining the aggregated information with its own information in the previous layer. For each vertex, its neighboring vertices are the vertices connected to it with edges, and its neighboring edges are the edges connected to it. As illustrated in Fig. 3(a)(b) where $K = 4$, for T_1, $R_1 \sim R_4$ are its neighboring vertices and edge (1,1) $\sim$ edge (1,4) are its neighboring edges, while for R_1, $T_1 \sim T_4$ are its neighboring vertices and edge (1,1) $\sim$ edge (4,1) are its neighboring edges.



(a) Neighboring vertices and edges of T_1 (b) Neighboring vertices and edges of R_1

(c) Neighboring edges of edge $(1,1)$ (d) Neighboring edges of edge $(1,2)$

FIGURE 3. Vertex-GNN (a) and (b) (When updating the representation of a vertex in blue color, the information of the vertices and edges with the same color is aggregated with the same weight.) and Edge-GNN (c) and (d) (When updating the representation of an edge with dashed lines, the information of the edges with the same color is aggregated with the same weight.)

For the Vertex-GNN learning over a heterogeneous graph, the hidden representation of the ith vertex with the τ_ith type in the lth layer, $\mathbf{d}_{i,\tau_i}^{(l)}$, is updated as follows [27],

$$\begin{aligned}&\textbf{Aggregate: } \mathbf{a}_{i,\tau_i}^{(l)} = \mathsf{PL}_{j\in\mathcal{N}(i)}\left(q(\mathbf{d}_{j,\tau_j}^{(l-1)}, \mathbf{e}_{ij}; \mathbf{W}_{\tau_i\tau_j\tau_{ij}}^{(l)})\right),\\ &\textbf{Combine: } \mathbf{d}_{i,\tau_i}^{(l)} = \mathsf{CB}\left(\mathbf{d}_{i,\tau_i}^{(l-1)}, \mathbf{a}_{i,\tau_i}^{(l)}; \mathbf{U}_{\tau_i}^{(l)}\right),\end{aligned} \tag{4}$$

where $\mathbf{a}_{i,\tau_i}^{(l)}$ is the aggregated output at the ith vertex, τ_i and τ_{ij} are respectively the types of the ith vertex and edge (i,j), $\mathcal{N}(i)$ is the set of neighboring vertices of the ith vertex, $\mathbf{e}_{ij}$ is the feature of edge (i,j), $q(\cdot)$, $\mathsf{PL}(\cdot)$ and $\mathsf{CB}(\cdot)$ are respectively the processing, pooling, and combination functions, and $\mathbf{W}_{\tau_i\tau_j\tau_{ij}}^{(l)}$ and $\mathbf{U}_{\tau_i}^{(l)}$ are weight matrices.

The choices for the processing and combination functions are flexible [11]. To ensure a GNN satisfying the PE property of a learned policy, the pooling function should satisfy the commutative law, e.g., sum-pooling $\sum_{k=1}^{K}(\cdot)$, mean-pooling $\frac{1}{K}\sum_{k=1}^{K}(\cdot)$, and max-pooling $\max_{k=1}^{K}(\cdot)$.

Remark 1. We consider simple pooling, processing, and combination functions as in [5], [9], [19], [21] for easy analysis, where $\mathsf{PL}(\cdot)$ is the sum-pooling function, $q(\cdot)$ is a linear function, and $\mathsf{CB}(\cdot)$ is a FNN without hidden layer (i.e., a linear function cascaded with an activation function), unless otherwise specified. The GNNs with the three functions are called vanilla GNNs.

The vanilla GNN updating vertex representations for learning the link scheduling policy over $\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$ is referred to as vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$, where the active state x_i is defined as the action of vertex T_i. Since there are two types of vertices in $\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$, the GNN respectively updates the hidden representations of the vertices of each type in each layer.

From (4), the hidden representations of T_i and R_i in the lth layer of the vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$, $\mathbf{d}_{i,T}^{(l)}$ and $\mathbf{d}_{i,R}^{(l)}$, are updated as,

Update hidden representations of tx vertices:

$$\begin{aligned}\textbf{Aggregate: } \mathbf{a}_{i,T}^{(l)} &= \mathbf{Q}_{RS}^{(l)}\mathbf{d}_{i,R}^{(l-1)} + \mathbf{P}_{RS}^{(l)}\alpha_{ii}\\ &\quad + \sum_{j=1,j\neq i}^{K}\left(\mathbf{Q}_{RI}^{(l)}\mathbf{d}_{j,R}^{(l-1)} + \mathbf{P}_{RI}^{(l)}\alpha_{ij}\right),\end{aligned} \tag{5a}$$

$$\textbf{Combine: } \mathbf{d}_{i,T}^{(l)} = \sigma\left(\mathbf{U}_{T}^{(l)}\mathbf{d}_{i,T}^{(l-1)} + \mathbf{a}_{i,T}^{(l)}\right),$$

Update hidden representations of rx vertices:

$$\begin{aligned}\textbf{Aggregate: } \mathbf{a}_{i,R}^{(l)} &= \mathbf{Q}_{TS}^{(l)}\mathbf{d}_{i,T}^{(l-1)} + \mathbf{P}_{TS}^{(l)}\alpha_{ii}\\ &\quad + \sum_{j=1,j\neq i}^{K}\left(\mathbf{Q}_{TI}^{(l)}\mathbf{d}_{j,T}^{(l-1)} + \mathbf{P}_{TI}^{(l)}\alpha_{ji}\right),\end{aligned} \tag{5b}$$

$$\textbf{Combine: } \mathbf{d}_{i,R}^{(l)} = \sigma\left(\mathbf{U}_{R}^{(l)}\mathbf{d}_{i,R}^{(l-1)} + \mathbf{a}_{i,R}^{(l)}\right),$$

where $\mathbf{a}_{i,T}^{(l)}$ and $\mathbf{a}_{i,R}^{(l)}$ are respectively the aggregated outputs at T_i and R_i, $\mathbf{Q}_{RS}^{(l)}$, $\mathbf{P}_{RS}^{(l)}$, $\mathbf{Q}_{RI}^{(l)}$, and $\mathbf{P}_{RI}^{(l)}$ are respectively the weight matrices for processing the information of R_i, edge (i,i), $R_j, j \neq i$, and edges $(i,j), j \neq i$, $\mathbf{U}_{T}^{(l)}$ is the weight matrix for combining when updating T_i, $\sigma(\cdot)$ is an activation function, $\mathbf{Q}_{TS}^{(l)}$, $\mathbf{P}_{TS}^{(l)}$, $\mathbf{Q}_{TI}^{(l)}$, $\mathbf{P}_{TI}^{(l)}$, and $\mathbf{U}_{R}^{(l)}$ in (5b) are respectively the weight matrices used for processing and combining when updating R_i.

In the input layer (i.e., $l = 0$), we set $d_{i,T}^{(0)} = 0$ and $d_{i,R}^{(0)} = 0$, because the vertices in $\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$ have no features. The input of the GNN is $\boldsymbol{\alpha}$, which is composed of the features of all edges. The output of the GNN is $[d_{1,T}^{(L)}, \cdots, d_{K,T}^{(L)}]^{\mathsf{T}} \triangleq [\hat{x}_{1,T}, \cdots, \hat{x}_{K,T}]^{\mathsf{T}} \triangleq \hat{\mathbf{x}}_T$, which is composed of the learned actions taken on all the tx vertices, where L is the number of layers of the GNN. Denote the policy learned by the vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ as $\hat{\mathbf{x}}_T = G_{\mathsf{ls,v}}^{\mathsf{het}}(\boldsymbol{\alpha})$. It is not hard to show that the learned policy is joint-PE to $\boldsymbol{\alpha}$.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

Remark 2. In [5], [6], [15], Vertex-GNNs were used to learn the link scheduling and power control policies over directed homogeneous graphs with the same topology as $\mathcal{G}_{\mathsf{ls}}^{\mathsf{hom}}$ in Fig. 2(a). To apply the update equation in (4), one can convert $\mathcal{G}_{\mathsf{ls}}^{\mathsf{hom}}$ into a heterogeneous graph, denoted as $\mathcal{G}_{\mathsf{ls,v}}^{\mathsf{undir}}$. When updating the representation of vertex D_i, its neighboring edges (i,j) and (j,i) $(j \neq i)$ are two types of edges, and its neighboring vertices are with two different types. Then, the representation of D_i in the lth layer, $\mathbf{d}_{i,V}^{(l)}$, can be obtained from (4), where $d_{i,V}^{(0)} = \alpha_{ii}$. The input of the GNN is $\boldsymbol{\alpha}$, which is composed of the features of all vertices and edges. The output of the GNN is $[d_{1,V}^{(L)}, \cdots, d_{K,V}^{(L)}]^{\mathsf{T}} \triangleq [\hat{x}_{1,V}, \cdots, \hat{x}_{K,V}]^{\mathsf{T}} \triangleq \hat{\mathbf{x}}_V$, which includes the learned actions on all vertices. When the three functions in Remark 1 are used, the Vertex-GNN learning over $\mathcal{G}_{\mathsf{ls,v}}^{\mathsf{undir}}$ is referred to as vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$. It is not hard to show that the learned policy is joint-PE to $\boldsymbol{\alpha}$.

Remark 3. When using a Vertex-GNN to learn the precoding policy over $\mathcal{G}_{\mathsf{p}}^{\mathsf{het}}$, the hidden representations of antenna vertex A_i and user vertex U_j in the lth layer, $\mathbf{d}_{i,A}^{(l)}$ and $\mathbf{d}_{j,U}^{(l)}$, can be obtained from (4), where $d_{i,A}^{(0)} = d_{j,U}^{(0)} = 0$ because the vertices have no features. Since the actions are defined on edges, a read-out layer is required to map the vertex representations in the output layer into the actions. In particular, to map $\mathbf{d}_{i,A}^{(L)}$ and $\mathbf{d}_{j,U}^{(L)}$ into the action on edge (i,j), a FNN can be designed as $\hat{v}_{ij,V} = \mathsf{FNN}_{\mathsf{read}}^{\mathsf{het}}(\mathbf{d}_{i,A}^{(L)}, \mathbf{d}_{j,U}^{(L)})$, which is the same for $i = 1, \cdots, N, j = 1, \cdots, K$. The input and output of the GNN are respectively $\mathbf{H}$ and $[\hat{v}_{ij,V}]_{N\times K} \triangleq \hat{\mathbf{V}}_V$, which are composed of the features and the learned actions of all edges. When the three functions in Remark 1 are used, the GNN is referred to as vanilla Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$. It is not hard to show that the learned policy is 2D-PE to $\mathbf{H}$.

2) Edge-GNN

In Edge-GNN, the hidden representation of each edge is updated in each layer, by first aggregating information from its neighboring edges and neighboring vertices, and then combining with its own hidden representation in the previous layer. For edge (i,j), the ith and jth vertices are its neighboring vertices, and the edges connected by the ith and jth vertices are its neighboring edges.

The update equation of an Edge-GNN can be obtained from the update equation of a Vertex-GNN simply by switching the roles of the edges and vertices [28]. For the Edge-GNN learning over a heterogeneous graph, the hidden representation of edge (i,j) with the τ_{ij}th type in the lth layer, denoted as $\mathbf{d}_{ij,\tau_{ij}}^{(l)}$, is updated as follows,

$$\begin{aligned}\textbf{Aggregate: } \mathbf{a}_{ij,\tau_{ij}}^{(l)} &= \underset{\substack{m\in\mathcal{N}(i)/j\\ n\in\mathcal{N}(j)/i}}{\mathsf{PL}}\Big(q(\mathbf{d}_{im,\tau_{im}}^{(l-1)}, \mathbf{e}_{v_i}; \mathbf{W}_{\tau_{ij}\tau_{im}\tau_i}^{(l)}),\\ &\qquad q(\mathbf{d}_{nj,\tau_{nj}}^{(l-1)}, \mathbf{e}_{v_j}; \mathbf{W}_{\tau_{ij}\tau_{nj}\tau_j}^{(l)})\Big),\\ \textbf{Combine: } \mathbf{d}_{ij,\tau_{ij}}^{(l)} &= \mathsf{CB}\Big(\mathbf{d}_{ij,\tau_{ij}}^{(l-1)}, \mathbf{a}_{ij,\tau_{ij}}^{(l)}; \mathbf{U}_{\tau_{ij}}^{(l)}\Big),\end{aligned} \tag{6}$$

where $\mathbf{a}_{ij,\tau_{ij}}^{(l)}$ is the aggregated output at edge (i,j), the first and second processors are respectively used to process the information from the ith vertex and its connected edges and the jth vertex and its connected edges, $\mathcal{N}(i)/j$ is a set of neighboring vertices of the ith vertex except the jth vertex, $\mathbf{e}_{v_j}$ denotes the feature of the jth vertex, and $\mathbf{W}_{\tau_{ij}\tau_{nj}\tau_j}^{(l)}$ and $\mathbf{U}_{\tau_{ij}}^{(l)}$ are the weight matrices.

When an Edge-GNN is used for learning the link scheduling policy over $\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$ in Fig. 2(b), the actions are defined on the sig edges. When the pooling, processing, and combination functions in Remark 1 are used, the Edge-GNN is referred to as vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$.

Since there are two types of edges in $\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$, the GNN respectively updates the hidden representations of the edges of each type in each layer. Since the vertices have no features in $\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$, when updating the representation of each edge, only the information of its neighboring edges is aggregated. For edge (i,i) (say, edge $(1,1)$ in Fig. 3(c)), edge (i,j) and edge (j,i) $(j \neq i)$ respectively connected by T_i and R_i are its neighboring edges. For edge (i,j) (say, edge $(1,2)$ in Fig. 3(d)), edge (i,i) and edge (i,k) $(k \neq \{i,j\})$ connected by T_i are respectively its neighboring sig and int edges, and edge (j,j) and edge (k,j) connected by R_j are respectively its neighboring sig and int edges.

From (6), the hidden representations of edge (i,i) and edge (i,j) in the lth layer of the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$, $\mathbf{d}_{i,S}^{(l)}$ and $\mathbf{d}_{ij,I}^{(l)}$, are updated as follows,

Update hidden representations of sig edges:

$$\textbf{Aggregate: } \mathbf{a}_{i,S}^{(l)} = \sum_{k=1,k\neq i}^{K} \mathbf{Q}_T^{(l)} \mathbf{d}_{ik,I}^{(l-1)} + \sum_{k=1,k\neq i}^{K} \mathbf{Q}_R^{(l)} \mathbf{d}_{ki,I}^{(l-1)}, \tag{7a}$$

$$\textbf{Combine: } \mathbf{d}_{i,S}^{(l)} = \sigma_S\Big(\mathbf{U}_S^{(l)} \mathbf{d}_{i,S}^{(l-1)} + \mathbf{a}_{i,S}^{(l)}\Big),$$

Update hidden representations of int edges:

$$\begin{aligned}\textbf{Aggregate: } \mathbf{a}_{ij,I}^{(l)} &= \mathbf{Q}_{ST}^{(l)} \mathbf{d}_{i,S}^{(l-1)} + \sum_{k=1,k\neq\{i,j\}}^{K} \mathbf{Q}_{IT}^{(l)} \mathbf{d}_{ik,I}^{(l-1)}\\ &\quad + \mathbf{Q}_{SR}^{(l)} \mathbf{d}_{j,S}^{(l-1)} + \sum_{k=1,k\neq\{i,j\}}^{K} \mathbf{Q}_{IR}^{(l)} \mathbf{d}_{kj,I}^{(l-1)},\\ \textbf{Combine: } \mathbf{d}_{ij,I}^{(l)} &= \sigma_I\Big(\mathbf{U}_I^{(l)} \mathbf{d}_{ij,I}^{(l-1)} + \mathbf{a}_{ij,I}^{(l)}\Big),\end{aligned} \tag{7b}$$

where $\mathbf{a}_{i,S}^{(l)}$ and $\mathbf{a}_{ij,I}^{(l)}$ are respectively the aggregated outputs at edge (i,i) and edge (i,j), $\mathbf{Q}_T^{(l)}$ and $\mathbf{Q}_R^{(l)}$ are respectively the weight matrices for processing the information of the neighboring int edges of the sig edge (i,i) connected by T_i and R_i, $\mathbf{Q}_{ST}^{(l)}$ and $\mathbf{Q}_{IT}^{(l)}$ are respectively the weight matrices for processing the information of the neighboring sig and int edges of the int edge (i,j) connected by T_i, $\mathbf{Q}_{SR}^{(l)}$ and $\mathbf{Q}_{IR}^{(l)}$ are respectively used for processing the information from neighboring sig and int edges of the int edge (i,j) connected by R_j, $\mathbf{U}_S^{(l)}$ and $\mathbf{U}_I^{(l)}$ are respectively the weight

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

matrices for combining the information of the sig and int edges, and $\sigma_S(\cdot)$ and $\sigma_I(\cdot)$ are activation functions.

In the input layer, $d_{i,S}^{(0)} = \alpha_{ii}$ and $d_{ij,I}^{(0)} = \alpha_{ij}$. The input of the GNN is $\boldsymbol{\alpha}$, which consists of the features of all edges. The output of the GNN is $[d_{1,S}^{(L)}, \cdots, d_{K,S}^{(L)}]^{\mathsf{T}} \triangleq [\hat{x}_{1,S}, \cdots, \hat{x}_{K,S}]^{\mathsf{T}} \triangleq \hat{\mathbf{x}}_S$, which consists of the learned actions on all the sig edges. The policy learned by the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ is denoted as $\hat{\mathbf{x}}_S = G_{\mathsf{ls,e}}^{\mathsf{het}}(\boldsymbol{\alpha})$, which is easily shown as joint-PE to $\boldsymbol{\alpha}$.

Remark 4. When using the Edge-GNN to learn the link scheduling policy over $\mathcal{G}_{\mathsf{ls}}^{\mathsf{hom}}$ with the update equations in (6), $\mathcal{G}_{\mathsf{ls}}^{\mathsf{hom}}$ needs to be converted into a heterogeneous graph, denoted as $\mathcal{G}_{\mathsf{ls,e}}^{\mathsf{undir}}$. For the directed edge (i,j), its neighboring vertices D_i and D_j are two types of vertices, its neighboring edges (i,k) and (j,k) are one type of edges while edges (k,i) and (k,j) are another type of edges. Then, the hidden representation of edge (i,j) in the lth layer, $\mathbf{d}_{ij,\mathsf{ls}}^{(l)}$, can be obtained from (6). Since the actions are defined on vertices in $\mathcal{G}_{\mathsf{ls}}^{\mathsf{hom}}$, a read-out layer is required to map the representations of edges in the output layer to the action on each vertex. For example, a FNN layer can be used that is shared among $D_i, i = 1, \cdots, K$, which can be designed as $\hat{x}_{i,E} = \mathsf{FNN}_{\mathsf{read}}^{\mathsf{undir}}\Big(\sum_{k=1,k\neq i}^{K} \mathbf{d}_{ik,\mathsf{ls}}^{(L)}, \sum_{k=1,k\neq i}^{K} \mathbf{d}_{ki,\mathsf{ls}}^{(L)}\Big)$. The input of the GNN is $\boldsymbol{\alpha}$, which is composed of the features of all the vertices and edges. The output of the GNN is $[\hat{x}_{1,E}, \cdots, \hat{x}_{K,E}] \triangleq \hat{\mathbf{x}}_E$, consisting of the learned actions on all vertices. When using the processing, pooling, and combination functions in Remark 1, the GNN is referred to as vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$. It is not hard to show that the learned policy is joint-PE to $\boldsymbol{\alpha}$.

Remark 5. In [19], the precoding policy was learned with a vanilla Edge-GNN over $\mathcal{G}_{\mathsf{p}}^{\mathsf{het}}$, where the hidden representation of edge (i,j) in the lth layer, $\mathbf{d}_{ij,E}^{(l)}$, can be obtained from (6) with the three functions in Remark 1. This GNN is referred to as vanilla Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$. The input and output of the GNN are respectively $\mathbf{H}$ and $[\mathbf{d}_{ij,E}^{(L)}]_{N\times K}$, which are composed of the features and the learned actions of all edges. It was shown that the learned policy is 2D-PE to $\mathbf{H}$.

The above-mentioned GNNs and the features and actions of graphs for each GNN are summarized in Table 2 and Table 3, respectively.

C. Expressive Power of the Vanilla GNNs

In what follows, we analyze the expressive power of the vanilla Vertex-GNNs and vanilla Edge-GNNs, by observing whether or not a GNN can output different actions when inputting different channel matrices. Without loss of generality, we assume that $L \geq 3$.

We first define several notations to be used in the sequel.

$\alpha_{\mathrm{R/i}} \triangleq \sum_{j=1,j\neq i}^{K} \alpha_{ij}$ and $\alpha_{\mathrm{C/i}} \triangleq \sum_{j=1,j\neq i}^{K} \alpha_{ji}$, which are the summations of the ith row of the channel matrix $\boldsymbol{\alpha}$ without α_{ii} and the ith column of $\boldsymbol{\alpha}$ without α_{ii}, respectively.

$H_{\mathrm{R,i}} \triangleq \sum_{j=1}^{K} h_{ij}$ and $H_{\mathrm{C,i}} \triangleq \sum_{j=1}^{N} h_{ji}$, which are the summations of the ith row and column of the channel matrix $\mathbf{H}$, respectively.

$\mathcal{A}_{\mathrm{CR}} \triangleq \{\alpha_{\mathrm{R}/1}, \cdots, \alpha_{\mathrm{R/K}}, \alpha_{\mathrm{C}/1}, \cdots, \alpha_{\mathrm{C/K}}\}$, which is the set composing of $\alpha_{\mathrm{R/i}}$ and $\alpha_{\mathrm{C/i}}, i = 1, \cdots, K$.

$\mathcal{A}_{\mathrm{diag}} \triangleq \{\alpha_{11}, \cdots, \alpha_{KK}\}$, which is the set composing of all the diagonal elements in $\boldsymbol{\alpha}$.

$\mathcal{A}_{\mathrm{ind}} \triangleq \{\alpha_{ij}, i,j = 1, \cdots, K\}$, which is the set composing of all the elements in $\boldsymbol{\alpha}$.

$\mathcal{H}_{\mathrm{ind}} \triangleq \{h_{ij}, i = 1, \cdots, N, j = 1, \cdots, K\}$, which is the set composing of all elements in $\mathbf{H}$.

$f(\cdot)$ with different super- and sub-scripts are non-linear functions.

1) Link Scheduling Policy

For the vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$, by respectively substituting $\mathbf{a}_{i,T}^{(l)}$ and $\mathbf{a}_{i,R}^{(l)}$ into $\mathbf{d}_{i,T}^{(l)}$ and $\mathbf{d}_{i,R}^{(l)}$ in (5), the hidden representations of T_i and R_i in the lth layer can be respectively updated as,

$$\mathbf{d}_{i,T}^{(l)} = \sigma\Big(\mathbf{U}_T^{(l)}\mathbf{d}_{i,T}^{(l-1)} + \mathbf{Q}_{RS}^{(l)}\mathbf{d}_{i,R}^{(l-1)} + \mathbf{Q}_{RI}^{(l)}\sum_{j=1,j\neq i}^{K}\mathbf{d}_{j,R}^{(l-1)} + \mathbf{P}_{RS}^{(l)}\alpha_{ii} + \mathbf{P}_{RI}^{(l)}\alpha_{\mathrm{R/i}}\Big), \tag{8a}$$

$$\mathbf{d}_{i,R}^{(l)} = \sigma\Big(\mathbf{U}_R^{(l)}\mathbf{d}_{i,R}^{(l-1)} + \mathbf{Q}_{TS}^{(l)}\mathbf{d}_{i,T}^{(l-1)} + \mathbf{Q}_{TI}^{(l)}\sum_{j=1,j\neq i}^{K}\mathbf{d}_{j,T}^{(l-1)} + \mathbf{P}_{TS}^{(l)}\alpha_{ii} + \mathbf{P}_{TI}^{(l)}\alpha_{\mathrm{C/i}}\Big). \tag{8b}$$

Since $d_{i,T}^{(0)} = d_{i,R}^{(0)} = 0$, when $l = 1$, from (8) we have

$$\mathbf{d}_{i,T}^{(1)} = \sigma\Big(\mathbf{P}_{RS}^{(1)}\alpha_{ii} + \mathbf{P}_{RI}^{(1)}\alpha_{\mathrm{R/i}}\Big) \triangleq f_T^{(1)}(\alpha_{ii}, \alpha_{\mathrm{R/i}}), \tag{9a}$$

$$\mathbf{d}_{i,R}^{(1)} = \sigma\Big(\mathbf{P}_{TS}^{(1)}\alpha_{ii} + \mathbf{P}_{TI}^{(1)}\alpha_{\mathrm{C/i}}\Big) \triangleq f_R^{(1)}(\alpha_{ii}, \alpha_{\mathrm{C/i}}). \tag{9b}$$

TABLE 2. Vertex- and Edge-GNNs learned over the graphs in Fig.2

GNNs			Policies	Graphs
Vertex-GNNs	Vertex-GNN$_{\mathsf{ls}}$	Vertex-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$	Link scheduling/power control	$\mathcal{G}_{\mathsf{ls,v}}^{\mathsf{undir}}$, converted from $\mathcal{G}_{\mathsf{ls}}^{\mathsf{hom}}$ in Fig.2(a)
		Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$		$\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$ in Fig.2(b)
	Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$		Precoding	$\mathcal{G}_{\mathsf{p}}^{\mathsf{het}}$ in Fig.2(c)
Edge-GNNs	Edge-GNN$_{\mathsf{ls}}$	Edge-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$	Link scheduling/power control	$\mathcal{G}_{\mathsf{ls,e}}^{\mathsf{undir}}$, converted from $\mathcal{G}_{\mathsf{ls}}^{\mathsf{hom}}$ in Fig.2(a)
		Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$		$\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$ in Fig.2(b)
	Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$		Precoding	$\mathcal{G}_{\mathsf{p}}^{\mathsf{het}}$ in Fig.2(c)

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

TABLE 3. Features and actions of the graphs for each GNN

Graphs[a]	Features		Actions		GNNs
	Vertex	Edge	Vertex	Edge	
$\mathcal{G}_{\mathsf{ls,v}}^{\mathsf{undir}}$	vertex D_i: α_{ii}	edge (i,j): α_{ij}	vertex D_i: x_i or p_i	-	Vertex-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$
$\mathcal{G}_{\mathsf{ls,e}}^{\mathsf{undir}}$					Edge-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$
$\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$	-	edge (i,j): α_{ij}	vertex T_i: x_i or p_i	-	Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$
			-	sig edge (i,i): x_i or p_i	Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$
$\mathcal{G}_{\mathsf{p}}^{\mathsf{het}}$	-	edge (i,j): h_{ij}	-	edge (i,j): v_{ij}	Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$
					Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$

[a] $\mathcal{G}_{\mathsf{ls,v}}^{\mathsf{undir}}$ and $\mathcal{G}_{\mathsf{ls,e}}^{\mathsf{undir}}$ are the converted heterogeneous graphs from $\mathcal{G}_{\mathsf{ls}}^{\mathsf{hom}}$, as explained in Remark 2 and Remark 4.

When $l=2$, by substituting $\mathbf{d}_{i,T}^{(1)}$ and $\mathbf{d}_{i,R}^{(1)}$ in (9) into (8), it is not hard to derive

$$\mathbf{d}_{i,T}^{(2)} \triangleq f_{i,T}^{(2)}(\alpha_{11},\cdots,\alpha_{KK},\alpha_{\mathrm{C}/1},\cdots,\alpha_{\mathrm{C/K}},\alpha_{\mathrm{R/i}}), \quad (10a)$$

$$\mathbf{d}_{i,R}^{(2)} \triangleq f_{i,R}^{(2)}(\alpha_{11},\cdots,\alpha_{KK},\alpha_{\mathrm{R}/1},\cdots,\alpha_{\mathrm{R/K}},\alpha_{\mathrm{C/i}}). \quad (10b)$$

Similarly, the action taken over T_i can be derived as,

$$\begin{aligned}\hat{x}_{i,T} &\triangleq f_{i,T}(\alpha_{11},\cdots,\alpha_{KK},\alpha_{\mathrm{C}/1},\cdots,\alpha_{\mathrm{C/K}},\\ &\qquad \alpha_{\mathrm{R}/1},\cdots,\alpha_{\mathrm{R/K}}) \qquad (11)\\ &= f_{i,T}(\mathcal{A}_{\mathrm{diag}},\mathcal{A}_{\mathrm{CR}}).\end{aligned}$$

It is shown that the information of interference channel gains $\alpha_{ij}, i\neq j$ is lost after the aggregation in the update equation of the vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$.

Analogously, for the vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$, the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$, and the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$, the actions taken over the ith vertex or the ith sig edge can respectively be expressed as,

$$\begin{aligned}\hat{x}_{i,V} &\triangleq f_{i,V}(\mathcal{A}_{\mathrm{diag}},\mathcal{A}_{\mathrm{CR}}),\\ \hat{x}_{i,S} &\triangleq f_{i,S}(\mathcal{A}_{ind}), \qquad (12)\\ \hat{x}_{i,E} &\triangleq f_{i,E}(\mathcal{A}_{ind}).\end{aligned}$$

Denote the outputs of the vanilla Vertex-GNN$_{\mathsf{ls}}$ (i.e., Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ or Vertex-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$) as $\hat{\mathbf{x}}_{[1]}$ and $\hat{\mathbf{x}}_{[2]}$ with two different inputs $\boldsymbol{\alpha}_{[1]} = [\alpha_{[1],ij}]_{K\times K}$ and $\boldsymbol{\alpha}_{[2]} = [\alpha_{[2],ij}]_{K\times K}$, respectively. From (11) and (12), we can obtain the following observation.

Observation 1: $\hat{\mathbf{x}}_{[1]} = \hat{\mathbf{x}}_{[2]}$ if the elements in $\boldsymbol{\alpha}_{[1]}$ and $\boldsymbol{\alpha}_{[2]}$ satisfy the following conditions: (1) $\alpha_{[1],11} = \alpha_{[2],11},\cdots,\alpha_{[1],KK} = \alpha_{[2],KK}$, (2) $\alpha_{[1],\mathrm{C}/1} = \alpha_{[2],\mathrm{C}/1},\cdots,\alpha_{[1],\mathrm{C/K}} = \alpha_{[2],\mathrm{C/K}}$, and (3) $\alpha_{[1],\mathrm{R}/1} = \alpha_{[2],\mathrm{R}/1},\cdots,\alpha_{[1],\mathrm{R/K}} = \alpha_{[2],\mathrm{R/K}}$.

These conditions can be rewritten as a group of linear equations

$$\begin{aligned}\alpha_{[1],ii} &= \alpha_{[2],ii},\\ \sum_{j=1,j\neq i}^{K}\alpha_{[1],ji} &= \sum_{j=1,j\neq i}^{K}\alpha_{[2],ji},\\ \sum_{j=1,j\neq i}^{K}\alpha_{[1],ij} &= \sum_{j=1,j\neq i}^{K}\alpha_{[2],ij},\\ i &= 1,\cdots,K,\end{aligned} \qquad (13)$$

which consists of $3K$ equations and $2K^2$ variables. When $K\geq 3$, the number of variables is larger than the number of equations, and hence there are infinite solutions to these equations, i.e., there are infinite numbers of $\boldsymbol{\alpha}_{[1]}$ and $\boldsymbol{\alpha}_{[2]}$ satisfying the conditions.

The observation indicates that the vanilla Vertex-GNN$_{\mathsf{ls}}$ for learning the link scheduling policy $\mathbf{x}^* = F_{\mathsf{ls}}(\boldsymbol{\alpha})$ is unable to differentiate all channel matrices. When the pooling function in a vanilla Vertex-GNN$_{\mathsf{ls}}$ is replaced by mean-pooling or max-pooling, we can also find channel matrices that the GNN cannot differentiate.

Recalling that $\boldsymbol{\alpha}\in\mathbb{R}^{K\times K}$ and $\mathbf{x}^*\in\mathbb{I}^{K\times 1}$, the scheduling policy $\mathbf{x}^* = F_{\mathsf{ls}}(\boldsymbol{\alpha})$ is a many-to-one mapping where the channel matrix is compressed by the mapping. However, the mappings learned by a vanilla Vertex-GNN$_{\mathsf{ls}}$ may not be the same as the scheduling policy, because $F_{\mathsf{ls}}(\boldsymbol{\alpha}_{[1]}) = F_{\mathsf{ls}}(\boldsymbol{\alpha}_{[2]})$ does not necessarily hold when $\boldsymbol{\alpha}_{[1]}$ and $\boldsymbol{\alpha}_{[2]}$ satisfy the three conditions, as to be shown via simulations later. As a consequence, the vanilla Vertex-GNN$_{\mathsf{ls}}$ cannot well learn the link scheduling policy due to the information loss.

By contrast, the vanilla Edge-GNN$_{\mathsf{ls}}$ (i.e., Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ or Edge-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$) does not incur the information loss, since it can differentiate the channel matrices resulting in different optimization variables. This can be seen from (12), where the outputs of the vanilla Edge-GNN$_{\mathsf{ls}}$ (i.e., $\hat{x}_{i,S}$ and $\hat{x}_{i,E}$) depend on each individual channel gain (say α_{ij}) in $\boldsymbol{\alpha}$.

2) Precoding Policy

With similar derivations, we can show that $\mathbf{H}$ is compressed into $H_{\mathrm{R,i}}$ and $H_{\mathrm{C,j}}$, $i=1,\cdots,N$, $j=1,\cdots,K$ after the aggregation of the vanilla Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$ where the information in individual channel coefficients loses. As a result, the GNN is unable to differentiate the channel matrices $\mathbf{H}_{[1]} = [h_{[1],ij}]_{N\times K} \neq \mathbf{H}_{[2]} = [h_{[2],ij}]_{N\times K}$ that satisfy the following conditions: (1) $H_{[1],\mathrm{C},1} = H_{[2],\mathrm{C},1},\cdots,H_{[1],\mathrm{C,K}} = H_{[2],\mathrm{C,K}}$, (2) $H_{[1],\mathrm{R},1} = H_{[2],\mathrm{R},1},\cdots,H_{[1],\mathrm{R,N}} = H_{[2],\mathrm{R,N}}$, which can be expressed as the following group of equations,

$$\begin{aligned}\sum_{n=1}^{N}h_{[1],nj} &= \sum_{n=1}^{N}h_{[2],nj}, \quad j=1,\cdots,K,\\ \sum_{k=1}^{K}h_{[1],ik} &= \sum_{k=1}^{K}h_{[2],ik}, \quad i=1,\cdots,N.\end{aligned} \qquad (14)$$

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

In other words, when inputting $\mathbf{H}_{[1]}$ and $\mathbf{H}_{[2]}$, the outputs of the vanilla Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$ are identical. When the pooling function is mean-pooling or max-pooling, we can also find channel matrices that the vanilla Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$ cannot differentiate.

However, the precoding matrix depends on every channel coefficient h_{ij}. For example, when the signal-to-noise ratio (SNR) is very low, the optimal precoding matrix degenerates into K vectors each for maximal-ratio transmission (i.e., $v_{ij}^*/|v_{ij}^*| = h_{ij}^{\mathsf{H}}/|h_{ij}|$) [19]. As a result, the vanilla Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$ is unable to learn the optimal precoding policy.

By contrast, the vanilla Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$ can differentiate all channel matrices. The update equation of the vanilla Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$ designed in [19] is

$$\mathbf{d}_{ij,E}^{(l)} = \sigma(\mathbf{U}_E^{(l)}\mathbf{d}_{ij,E}^{(l-1)} + \sum_{m=1,m\neq j}^{K}\mathbf{Q}_A^{(l)}\mathbf{d}_{im,E}^{(l-1)} + \sum_{n=1,n\neq i}^{N}\mathbf{Q}_U^{(l)}\mathbf{d}_{nj,E}^{(l-1)}), \quad (15)$$

where $\mathbf{U}_E^{(l)}$ is the weight matrix for the combination, $\mathbf{Q}_A^{(l)}$ and $\mathbf{Q}_U^{(l)}$ are respectively the weight matrices for processing the information of the neighboring edges connected by A_i and U_j. In the input layer, $d_{ij,E}^{(0)} = h_{ij}$. The inputs of the GNN are the features of all edges, i.e., $[d_{ij,E}^{(0)}]_{N\times K} = [h_{ij}]_{N\times K} = \mathbf{H}$. The outputs of the GNN are the learned actions taken on all the edges, i.e., $[d_{ij,E}^{(L)}]_{N\times K} \triangleq [\hat{v}_{ij,E}]_{N\times K} \triangleq \hat{\mathbf{V}}_E$.

It can be derived from (15) that $\hat{v}_{ij,E} \triangleq f_{ij,E}(\mathcal{H}_{ind})$, since every channel coefficient (say h_{ij}) is combined individually when updating the edge representation with $l = 1$.

Remark 6. When considering other typical constraints, we can use the same way to analyze the expressive power of GNNs, but the input samples that GNNs cannot distinguish may differ.

IV. IMPACT OF PROCESSING AND COMBINATION FUNCTIONS ON EXPRESSIVE POWER

In this section, we analyze the impact of processing and combination functions on the expressive power of the Vertex-GNNs and Edge-GNNs for learning the policies.

We first analyze the impact of the linearity of processing and combination function, and then analyze the impact of the output dimensions of the two functions on Vertex-GNNs and Edge-GNNs.

Without the loss of generality, we assume that $L \geq 3$.

Denote $\alpha_{\mathsf{Tr}} \triangleq \sum_{j=1}^{K}\alpha_{jj}$. Denote $g(\cdot)$ with different super- and sub-scripts as linear functions.

A. Impact of Linearity

As analyzed in section C, the vanilla Vertex-GNNs (i.e., vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$, Vertex-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$, and Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$) cannot while the vanilla Edge-GNNs (i.e., vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$, Edge-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$, and Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$) can differentiate the channel matrices resulting in different optimization variables, where both classes of vanilla GNNs are with linear processors and non-linear combiners. In the sequel, we show that the expressive power of the Vertex-GNNs can be enhanced by using non-linear processors, and the strong expressive power of the Edges-GNNs comes from the non-linear combiners. We take the GNNs for learning the link scheduling policy as an example. The impact is the same on the GNNs for learning the power control and precoding policies.

1) Vertex-GNNs

We start by analyzing the expressive power of a degenerated vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ where the combination function becomes linear.

Linear processor and linear combinator: For Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$, $d_{i,T}^{(0)} = d_{i,R}^{(0)} = 0$. Then, after omitting the activation functions in (8), the hidden representations of T_i and R_i become,

$$\mathbf{d}_{i,T}^{(1)} = \mathbf{P}_{RS}^{(l)}\alpha_{ii} + \mathbf{P}_{RI}^{(1)}\alpha_{\mathrm{R/i}} \triangleq g_T^{(1)}(\alpha_{ii}, \alpha_{\mathrm{R/i}}), \quad (16a)$$

$$\mathbf{d}_{i,R}^{(1)} = \mathbf{P}_{TS}^{(l)}\alpha_{ii} + \mathbf{P}_{TI}^{(1)}\alpha_{\mathrm{C/i}} \triangleq g_R^{(1)}(\alpha_{ii}, \alpha_{\mathrm{C/i}}). \quad (16b)$$

By substituting $\mathbf{d}_{i,T}^{(1)}$ and $\mathbf{d}_{i,R}^{(1)}$ into (8), and again omitting the activation functions, we have,

$$\begin{aligned}\mathbf{d}_{i,T}^{(2)} =& \mathbf{U}_T^{(2)}g_T^{(1)}(\alpha_{ii}, \alpha_{\mathrm{R/i}}) + \mathbf{Q}_{RS}^{(2)}g_R^{(1)}(\alpha_{ii}, \alpha_{\mathrm{C/i}}) \\ &+ \mathbf{Q}_{RI}^{(2)}\sum_{j=1,j\neq i}^{K} g_R^{(1)}(\alpha_{jj}, \alpha_{\mathrm{C/j}}) + \mathbf{P}_{RS}^{(2)}\alpha_{ii} + \mathbf{P}_{RI}^{(2)}\alpha_{\mathrm{R/i}} \\ \overset{(a)}{=}& \mathbf{U}_T^{(2)}g_T^{(1)}(\alpha_{ii}, \alpha_{\mathrm{R/i}}) + \mathbf{Q}_{RS}^{(2)}g_R^{(1)}(\alpha_{ii}, \alpha_{\mathrm{C/i}}) \\ &+ \mathbf{Q}_{RI}^{(2)}g_R^{(1)}(\alpha_{\mathrm{Tr}} - \alpha_{ii}, \|\boldsymbol{\alpha}\| - \alpha_{\mathrm{C/i}} - \alpha_{\mathrm{Tr}}) \\ &+ \mathbf{P}_{RS}^{(2)}\alpha_{ii} + \mathbf{P}_{RI}^{(2)}\alpha_{\mathrm{R/i}} \\ \triangleq& g_T^{(2)}(\alpha_{ii}, \alpha_{\mathrm{Tr}}, \alpha_{\mathrm{R/i}}, \alpha_{\mathrm{C/i}}, \|\boldsymbol{\alpha}\|),\end{aligned} \quad (17a)$$

$$\begin{aligned}\mathbf{d}_{i,R}^{(2)} =& \mathbf{U}_R^{(2)}g_R^{(1)}(\alpha_{ii}, \alpha_{\mathrm{C/i}}) + \mathbf{Q}_{TS}^{(2)}g_T^{(1)}(\alpha_{ii}, \alpha_{\mathrm{R/i}}) \\ &+ \mathbf{Q}_{IT}^{(2)}\sum_{j=1,j\neq i}^{K} g_T^{(1)}(\alpha_{jj}, \alpha_{\mathrm{R/j}}) + \mathbf{P}_{TS}^{(2)}\alpha_{ii} + \mathbf{P}_{TI}^{(2)}\alpha_{\mathrm{C/i}} \\ \overset{(a)}{=}& \mathbf{U}_R^{(2)}g_R^{(1)}(\alpha_{ii}, \alpha_{\mathrm{C/i}}) + \mathbf{Q}_{TS}^{(2)}g_T^{(1)}(\alpha_{ii}, \alpha_{\mathrm{R/i}}) \\ &+ \mathbf{Q}_{TI}^{(2)}g_T^{(1)}(\alpha_{\mathrm{Tr}} - \alpha_{ii}, \|\boldsymbol{\alpha}\| - \alpha_{\mathrm{R/i}} - \alpha_{\mathrm{Tr}}) \\ &+ \mathbf{P}_{TS}^{(2)}\alpha_{ii} + \mathbf{P}_{TI}^{(2)}\alpha_{\mathrm{C/i}} \\ \triangleq& g_R^{(2)}(\alpha_{ii}, \alpha_{\mathrm{Tr}}, \alpha_{\mathrm{R/i}}, \alpha_{\mathrm{C/i}}, \|\boldsymbol{\alpha}\|),\end{aligned} \quad (17b)$$

where (a) in both (17a) and (17b) is obtained by exchanging the operation order of the linear function and the summation function, i.e., $\sum g_T^{(1)}(\cdot) = g_T^{(1)}(\sum(\cdot))$ and $\sum g_R^{(1)}(\cdot) = g_R^{(1)}(\sum(\cdot))$.

With similar derivations, we can obtain that the action of vertex T_i is a linear function of the input features, i.e.,

$$\hat{x}_{i,T} \triangleq g_T(\alpha_{ii}, \alpha_{\mathrm{Tr}}, \alpha_{\mathrm{R/i}}, \alpha_{\mathrm{C/i}}, \|\boldsymbol{\alpha}\|), \quad (18)$$

which does not depend on α_{ij} $(i \neq j)$. This indicates that the degenerated vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ cannot distinguish the individual interference channel gains in $\boldsymbol{\alpha}$. As a result, the GNN may yield the same action for different input features $\boldsymbol{\alpha}_{[1]}$ and $\boldsymbol{\alpha}_{[2]}$.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

Remark 7. Analogously, when the combination functions are linear, the action of vertex D_i of the degenerated vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$ (i.e., $\hat{x}_{i,V}$), the action of vertex D_i of the degenerated vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$ (i.e., $\hat{x}_{i,E}$), and the action of sig edge (i,i) of the degenerated vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ (i.e., $\hat{x}_{i,S}$) can also be expressed in the form as (18).

Linear processor and non-linear combinator: From (4) and (5), we can see that four linear functions with different weight matrices are required in the vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$. In particular, the processing functions are respectively used for (a) tx vertex aggregating information from rx vertex and sig edge, (b) tx vertex aggregating information from rx vertex and inf edge, (c) rx vertex aggregating information from tx vertex and sig edge, and (d) rx vertex aggregating information from tx vertex and inf edge, which are respectively

$$\begin{aligned} q(\mathbf{d}_{i,R}^{(l-1)},\alpha_{ii};\mathbf{W}_{RS}^{(l)}) &= \mathbf{Q}_{RS}^{(l)}\mathbf{d}_{i,R}^{(l-1)}+\mathbf{P}_{RS}^{(l)}\alpha_{ii},\\ q(\mathbf{d}_{j,R}^{(l-1)},\alpha_{ij};\mathbf{W}_{RI}^{(l)}) &= \mathbf{Q}_{RI}^{(l)}\mathbf{d}_{j,R}^{(l-1)}+\mathbf{P}_{RI}^{(l)}\alpha_{ij},\\ q(\mathbf{d}_{i,T}^{(l-1)},\alpha_{ii};\mathbf{W}_{TS}^{(l)}) &= \mathbf{Q}_{TS}^{(l)}\mathbf{d}_{i,T}^{(l-1)}+\mathbf{P}_{TS}^{(l)}\alpha_{ii},\\ q(\mathbf{d}_{j,T}^{(l-1)},\alpha_{ji};\mathbf{W}_{TI}^{(l)}) &= \mathbf{Q}_{TI}^{(l)}\mathbf{d}_{j,T}^{(l-1)}+\mathbf{P}_{TI}^{(l)}\alpha_{ji}. \end{aligned} \tag{19}$$

Then, the aggregated outputs of T_i and R_i in (5) can be respectively rewritten as,

$$\begin{aligned} \mathbf{a}_{i,T}^{(l)} =& q(\mathbf{d}_{i,R}^{(l-1)},\alpha_{ii};\mathbf{W}_{RS}^{(l)})+\sum_{j=1,j\neq i}^{K} q(\mathbf{d}_{j,R}^{(l-1)},\alpha_{ij};\mathbf{W}_{RI}^{(l)})\\ =& \mathbf{Q}_{RS}^{(l)}\mathbf{d}_{i,R}^{(l-1)}+\mathbf{Q}_{RI}^{(l)}\sum_{j=1,j\neq i}^{K}\mathbf{d}_{j,R}^{(l-1)}+\mathbf{P}_{RS}^{(l)}\alpha_{ii}+\mathbf{P}_{RI}^{(l)}\alpha_{\mathrm{R/i}}, \end{aligned} \tag{20a}$$

$$\begin{aligned} \mathbf{a}_{i,R}^{(l)} =& q(\mathbf{d}_{i,T}^{(l-1)},\alpha_{ii};\mathbf{W}_{TS}^{(l)})+\sum_{j=1,j\neq i}^{K} q(\mathbf{d}_{j,T}^{(l-1)},\alpha_{ji};\mathbf{W}_{TI}^{(l)})\\ =& \mathbf{Q}_{TS}^{(l)}\mathbf{d}_{i,T}^{(l-1)}+\mathbf{Q}_{TI}^{(l)}\sum_{j=1,j\neq i}^{K}\mathbf{d}_{j,T}^{(l-1)}+\mathbf{P}_{TS}^{(l)}\alpha_{ii}+\mathbf{P}_{TI}^{(l)}\alpha_{\mathrm{C/i}}. \end{aligned} \tag{20b}$$

Since $d_{k,T}^{(0)}=d_{k,R}^{(0)}=0$, $\mathbf{a}_{i,T}^{(1)}$ and $\mathbf{d}_{i,T}^{(1)}$ depend on α_{ii} and $\alpha_{\mathrm{R/i}}$, and $\mathbf{a}_{i,R}^{(1)}$ and $\mathbf{d}_{i,R}^{(1)}$ depend on α_{ii} and $\alpha_{\mathrm{C/i}}$. With similar derivations, it can be shown that the outputs of the GNN depend on $\mathcal{A}_{\mathrm{diag}}$ and $\mathcal{A}_{\mathrm{CR}}$, which are respectively composed of α_{ii}, $\alpha_{\mathrm{R/i}}$ and $\alpha_{\mathrm{C/i}}, i=1,\cdots,K$. If $\sigma(\cdot)$ in the combination function is replaced by a FNN, then it is not hard to show that $\hat{x}_{i,T}$ has the same form as in (11). This means that the information of interference channel gains $\alpha_{ij}, i\neq j$ is lost after the linear processing. As a consequence, the GNN cannot distinguish $\boldsymbol{\alpha}_{[1]}$ and $\boldsymbol{\alpha}_{[2]}$.

Non-linear processor and linear/non-linear combiner: When the processors in the vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ are replaced by non-linear functions (say FNNs), the aggregated outputs of T_i and R_i after passing through the sum-pooling become,

$$\mathbf{a}_{i,T}^{(l)} = \mathsf{FNN}_{RS}\left(\mathbf{d}_{i,R}^{(l-1)},\alpha_{ii}\right)+\sum_{j=1,j\neq i}^{K}\mathsf{FNN}_{RI}\left(\mathbf{d}_{j,R}^{(l-1)},\alpha_{ij}\right), \tag{21a}$$

$$\mathbf{a}_{i,R}^{(l)} = \mathsf{FNN}_{TS}\left(\mathbf{d}_{i,T}^{(l-1)},\alpha_{ii}\right)+\sum_{j=1,j\neq i}^{K}\mathsf{FNN}_{TI}\left(\mathbf{d}_{j,T}^{(l-1)},\alpha_{ji}\right). \tag{21b}$$

Since $\sum\mathsf{FNN}(\cdot)\neq\mathsf{FNN}(\sum(\cdot))$ and $\mathbf{d}_{k,T}^{(0)}=\mathbf{d}_{k,R}^{(0)}=0$, $\mathbf{a}_{i,T}^{(1)}$ depends on $\mathcal{A}_{i*}\triangleq\{\alpha_{i1},\cdots,\alpha_{iK}\}$, and $\mathbf{a}_{i,R}^{(1)}$ depends on $\mathcal{A}_{*i}\triangleq\{\alpha_{1i},\cdots,\alpha_{Ki}\}$. After the combiner (no matter linear or non-linear), $\mathbf{d}_{i,T}^{(1)}$ and $\mathbf{d}_{i,R}^{(1)}$ respectively depend on $\mathcal{A}_{i*}$ and $\mathcal{A}_{*i}$. It can be shown with similar derivations that the outputs of the GNN depend on $\mathcal{A}_{\mathsf{ind}}$, which is the set of all the elements in $\boldsymbol{\alpha}$. In other words, the outputs of the GNN depend on α_{ij}, and hence the GNN can distinguish $\boldsymbol{\alpha}_{[1]}$ and $\boldsymbol{\alpha}_{[2]}$.

Similarly, we can show that the expressive power of Vertex-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$ is able to be improved by using FNN for processing, but cannot be improved by using FNN for combining.

2) Edge-GNNs

Since the outputs of all GNNs for link scheduling depend on $\alpha_{ii}, i=1,\cdots,K$ when the processing and combination functions are linear as in Remark 7, we only analyze whether they depend on individual interference channel gains in the following. According to Remark 7 and the analysis in Section C, it is the non-linear combination functions that help the vanilla Edge-GNN$_{\mathsf{ls}}$ distinguish individual interference channels. Since there are two types of combination functions in each layer of the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ as shown in (7), in the following we analyze which combiner helps the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ distinguish $\alpha_{ij}, i\neq j$.

Since $d_{i,S}^{(0)}=\alpha_{ii}$ and $d_{ij,I}^{(0)}=\alpha_{ij}$, the combination functions of the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ for updating the hidden representations of sig edge (i,i) and int edge (i,j) in the first layer can be obtained from (7a) and (7b) as,

$$\begin{aligned} \mathbf{d}_{i,S}^{(1)} &= \sigma_S\left(\mathbf{U}_S^{(1)}d_{i,S}^{(0)}+\mathbf{a}_{i,S}^{(1)}\right)\triangleq\mathsf{CB}_S^{(1)}\left(d_{i,S}^{(0)},\mathbf{a}_{i,S}^{(1)}\right)\\ &\overset{(a)}{=}\mathsf{CB}_S^{(1)}\left(\alpha_{ii},\sum_{k=1,k\neq i}^{K}\mathbf{Q}_T^{(1)}\alpha_{ik}+\sum_{k=1,k\neq i}^{K}\mathbf{Q}_R^{(1)}\alpha_{ki}\right) \qquad (22a)\\ &\overset{(b)}{=}\mathsf{CB}_S^{(1)}\left(\alpha_{ii},\mathbf{Q}_T^{(1)}\alpha_{\mathrm{R/i}}+\mathbf{Q}_R^{(1)}\alpha_{\mathrm{C/i}}\right), \end{aligned}$$

$$\begin{aligned} \mathbf{d}_{ij,I}^{(1)} &= \sigma_I\left(\mathbf{U}_I^{(1)}d_{ij,I}^{(0)}+\mathbf{a}_{ij,I}^{(1)}\right)\triangleq\mathsf{CB}_I^{(1)}\left(d_{ij,I}^{(0)},\mathbf{a}_{ij,I}^{(1)}\right)\\ &\overset{(a)}{=}\mathsf{CB}_I^{(1)}\Big(\alpha_{ij},\mathbf{Q}_{ST}^{(1)}\alpha_{ii}+\sum_{k=1,k\neq\{i,j\}}^{K}\mathbf{Q}_{IT}^{(1)}\alpha_{ik}\\ &\qquad+\mathbf{Q}_{SR}^{(1)}\alpha_{jj}+\sum_{k=1,k\neq\{i,j\}}^{K}\mathbf{Q}_{IR}^{(1)}\alpha_{kj}\Big) \qquad (22b)\\ &\overset{(b)}{=}\mathsf{CB}_I^{(1)}\Big(\alpha_{ij},\mathbf{Q}_{ST}^{(1)}\alpha_{ii}+\mathbf{Q}_{IT}^{(1)}\alpha_{\mathrm{R/i}}+\mathbf{Q}_{SR}^{(1)}\alpha_{jj}\\ &\qquad+\mathbf{Q}_{IR}^{(1)}\alpha_{\mathrm{C/j}}-(\mathbf{Q}_{IT}^{(1)}+\mathbf{Q}_{IR}^{(1)})\alpha_{ij}\Big)\\ &\triangleq J_{ij}(\alpha_{ij}), \end{aligned}$$

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

where (a) in (22a) or (22b) comes from substituting $d_{i,S}^{(0)}$ and $d_{ij,I}^{(0)}$ into $\mathbf{a}_{i,S}^{(1)}$ and $\mathbf{a}_{ij,I}^{(1)}$, (b) in (22a) or (22b) is obtained by exchanging the order of matrix multiplication and summation operations. $J_{ij}(\cdot)$ is expressed as a function of only α_{ij}, since we are concerned whether or not the output of GNN depends on individual interference channel gains.

If $\sigma_I(\cdot)$ is omitted, then $\mathsf{CB}_I^{(1)}(\cdot)$ and $J_{ij}(\cdot)$ in (22b) become linear functions. Then, when $l = 2$, the first term in (7a) becomes $\sum_{k=1,k\neq i}^{K} \mathbf{Q}_T^{(2)} \mathbf{d}_{ik,I}^{(1)} = \mathbf{Q}_T^{(2)} \sum_{k=1,k\neq i}^{K} J_{ik}(\alpha_{ik})$ that depends on $\alpha_{\mathrm{R/i}}$, and the second term in (7a) becomes $\sum_{k=1,k\neq i}^{K} \mathbf{Q}_R^{(2)} \mathbf{d}_{ki,I}^{(1)}$ that depends on $\alpha_{\mathrm{C/i}}$. Hence, $\mathbf{d}_{i,S}^{(2)}$ depends on $\alpha_{\mathrm{R/i}}$ and $\alpha_{\mathrm{C/i}}$. With similar analysis, it can be shown that the action on edge (i,i) (i.e., $\hat{x}_{i,S}$) still depends on $\mathcal{A}_{\mathrm{CR}}$, i.e., the corresponding Edge-GNN cannot distinguish $\alpha_{ij}, i \neq j$.

With $\sigma_I(\cdot)$, $\mathsf{CB}_I^{(1)}(\cdot)$ and $J_{ij}(\cdot)$ in (22b) are non-linear. Since $\sum J_{ij}(\cdot) \neq J_{ij}(\sum(\cdot))$, $\sum_{k=1,k\neq i}^{K} \mathbf{Q}_T^{(2)} \mathbf{d}_{ik,I}^{(1)}$ depends on $\alpha_{ik}, k \neq i$ rather than $\alpha_{\mathrm{R/i}}$, and $\sum_{k=1,k\neq i}^{K} \mathbf{Q}_R^{(2)} \mathbf{d}_{ki,I}^{(1)}$ depends on α_{ki} rather than $\alpha_{\mathrm{C/i}}$. Hence, $\mathbf{d}_{i,S}^{(2)}$ depends on $\mathcal{A}_{i*}$ and $\mathcal{A}_{*i}$ no matter if $\sigma_S(\cdot)$ is omitted. Similarly, we can show that the action of edge (i,i) depends on $\mathcal{A}_{\mathsf{ind}}$, i.e., the corresponding GNN can distinguish $\alpha_{ij}, i \neq j$.

With similar analysis, we can show that it is $\sigma_I(\cdot)$ (instead of $\sigma_S(\cdot)$) in the Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ that enables the GNN to distinguish $\alpha_{ij}, i \neq j$ and enhances its expressive power.

In a nutshell, non-linear processors help improve the expressive power of the vanilla Vertex-GNNs (i.e., Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$, Vertex-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$, and Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$). Non-linear combiners for updating the int edges (say $\sigma_I(\cdot)$) help the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ to distinguish $\alpha_{ij}, i \neq j$. Since there is only one type of combination functions in the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$ and Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$, the non-linearity of all combination functions helps improve the expressive power of the two Edge-GNNs. The expressive power of these GNNs is summarized in Table 4.

B. Impact of Dimension

When a Vertex-GNN learns the link scheduling or power control policy over $\mathcal{G}_{\mathsf{ls}}^{\mathsf{het}}$ that consists of $2K$ vertices, there are $2K$ update equations, where two of them for updating $\mathbf{d}_{i,T}^{(l)} \in \mathbb{R}^{M^{(l)}}$ and $\mathbf{d}_{i,R}^{(l)} \in \mathbb{R}^{M^{(l)}}$ are shown in (8) and $M^{(l)}$ is an integer that is a hyper-parameter. The representation of all vertices of the Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ in the lth layer is $[\mathbf{d}_{1,T}^{(l)}, \cdots, \mathbf{d}_{K,T}^{(l)}, \mathbf{d}_{1,R}^{(l)}, \cdots, \mathbf{d}_{K,R}^{(l)}] \in \mathbb{R}^{M^{(l)} \times 2K}$, whose dimension is higher than the dimension of action vector $\hat{\mathbf{x}}_T \in \mathbb{R}^K$. Analogously, the representation of all vertices of the Vertex-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$ in the lth layer is $[\mathbf{d}_{1,V}^{(l)}, \cdots, \mathbf{d}_{K,V}^{(l)}] \in \mathbb{R}^{M^{(l)} \times K}$, whose dimension is no less than the dimension of action vector $\hat{\mathbf{x}}_V \in \mathbb{R}^K$. When Edge-GNNs are used to learn the two policies, the dimension of the representation of all edges is higher than K. Since channel information is not compressed with the high representation dimension, the dimensions of the GNNs for these two policies do not affect their expressive power. In the sequel, we only analyze the impact of the dimensions for the GNNs learning the precoding policy.

When learning over $\mathcal{G}_{\mathsf{p}}^{\mathsf{het}}$ consisting of $N + K$ vertices and NK edges, the hidden representations in the lth layer of the Vertex-GNN are $[\mathbf{d}_{1,A}^{(l)}, \cdots, \mathbf{d}_{N,A}^{(l)}] \in \mathbb{R}^{M_A^{(l)} \times N}$ and $[\mathbf{d}_{1,U}^{(l)}, \cdots, \mathbf{d}_{K,U}^{(l)}] \in \mathbb{R}^{M_U^{(l)} \times N}$, and that of the Edge-GNN is $[\mathbf{d}_{11,E}^{(l)}, \cdots, \mathbf{d}_{NK,E}^{(l)}] \in \mathbb{R}^{M_E^{(l)} \times NK}$, where $M_A^{(l)}$, $M_U^{(l)}$, and $M_E^{(l)}$ are hyper-parameters respectively representing the dimensions of the hidden representations for each antenna vertex, each user vertex, and each edge. Since the precoding policy maps $\mathbf{H} \in \mathbb{C}^{N \times K}$ into $\mathbf{V}^* \in \mathbb{C}^{N \times K}$, the channel information will be lost if the vertex representation is with low dimension.

1) Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$

As shown in Fig. 4(a), the combination outputs of all antenna and user vertices in the lth layer are $\mathbf{d}_A^{(l)} \triangleq [\mathbf{d}_{1,A}^{(l)}, \cdots, \mathbf{d}_{N,A}^{(l)}] \in \mathbb{R}^{M_A^{(l)} \times N}$ and $\mathbf{d}_U^{(l)} \triangleq [\mathbf{d}_{1,U}^{(l)}, \cdots, \mathbf{d}_{K,U}^{(l)}] \in \mathbb{R}^{M_U^{(l)} \times K}$, the aggregation outputs of A_i and U_j in the lth layer are $\mathbf{a}_{i,A}^{(l)} \in \mathbb{R}^{M_{A,q}^{(l)}}$, and $\mathbf{a}_{j,U}^{(l)} \in \mathbb{R}^{M_{U,q}^{(l)}}$, respectively, where $M_{A,q}^{(l)}$ and $M_{U,q}^{(l)}$ are hyper-parameters.

The GNN can be expressed as composite functions $\varphi_{V;\Theta}^{O}(\varphi_{RV;\Theta}^{(L)}(\mathbf{H}))$, where $\varphi(\cdot)$ with sub-script Θ denotes the functions with trainable parameters, $\varphi_{RV;\Theta}^{(L)}(\mathbf{H})$ maps $\mathbf{H}$ (i.e., the input feature of the GNN) to $[\mathbf{d}_A^{(L)}, \mathbf{d}_U^{(L)}]$ (i.e., the

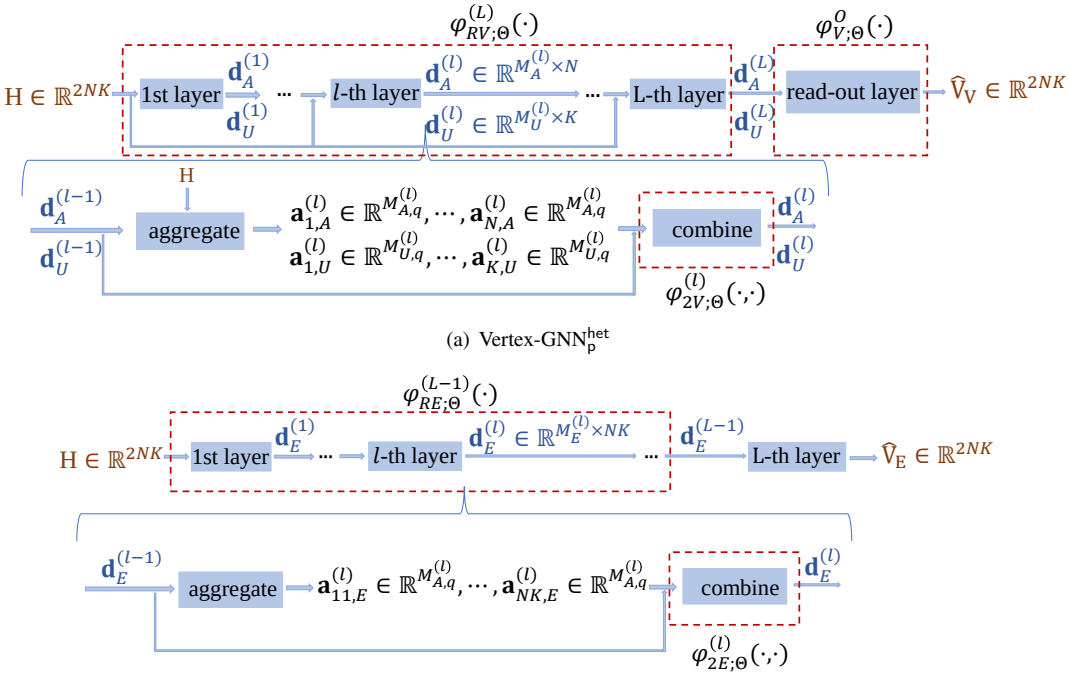
TABLE 4. Expressive power of GNNs

GNNs		Processing functions	Combination functions	Expressive power
Vertex-GNNs		linear	linear	weak
			non-linear	
		non-linear	linear	strong
			non-linear	
Edge-GNNs	Edge-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$	linear	non-linear	strong
	Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$			
	Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$	linear	CB_S is non-linear, CB_I is linear[a]	weak
			CB_S is linear or non-linear, CB_I is non-linear	strong

[a] CB_S and CB_I are respectively the combination functions for updating the hidden representations of sig and int edges.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872



(a) Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$

(b) Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$

FIGURE 4. Structure of (a) Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$ and (b) Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$.

vertex representations in the Lth layer), and $\varphi_{V;\Theta}^{O}(\cdot)$ maps $[\mathbf{d}_A^{(L)}, \mathbf{d}_U^{(L)}]$ to $\hat{\mathbf{V}}_V$ (i.e., the actions on edges). In order for the GNN not compressing channel information, $\varphi_{RV;\Theta}^{(L)}(\mathbf{H})$ should not compress the channel dimension. Otherwise, the GNN may not differentiate all channel matrices.

Since matrix $\mathbf{X} = [x_{ij}]_{m\times n}$ can be vectorized as a (mn)-dimensional vector $\overline{\mathbf{x}} \in \mathbb{R}^{mn}$, a block matrix $[\mathbf{X}_1, \mathbf{X}_2]$ with $\mathbf{X}_1 = [x_{1,ij}]_{m_1\times n_1}$ and $\mathbf{X}_2 = [x_{2,ij}]_{m_2\times n_2}$ seems able to be vectorized to a $(m_1+m_2)(n_1+n_2)$-dimensional vector, because the block matrix before vectorization can be expressed as $\begin{pmatrix} \mathbf{X}_1 & [0]_{m_1\times n_2} \\ [0]_{m_2\times n_1} & \mathbf{X}_2 \end{pmatrix}$. However, since zero matrices do not occupy space, the dimension of the vectorized block matrix is $(m_1n_1 + m_2n_2)$. Hence, the output dimension of $\varphi_{RV;\Theta}^{(L)}(\mathbf{H})$ is $(NM_A^{(L)} + KM_U^{(L)})$. $\mathbf{H} \in \mathbb{C}^{N\times K}$ can be vectorized as a $2NK$-dimensional real vector. In order for $\varphi_{RV;\Theta}^{(L)}(\mathbf{H})$ not compressing channel information, the output dimension of $\varphi_{RV;\Theta}^{(L)}(\mathbf{H})$ should be no less than the dimension of $\mathbf{H}$, i.e., $NM_A^{(L)} + KM_U^{(L)} \geq 2NK$.

$\varphi_{RV;\Theta}^{(L)}(\mathbf{H})$ can further be expressed as the following composite functions $\varphi_{RV;\Theta}^{(L)}(\mathbf{H}) \triangleq \varphi_{2V;\Theta}^{(L)}(\varphi_{RV;\Theta}^{(L-1)}(\mathbf{H}), \varphi_{AV;\Theta}^{(L)}(\mathbf{H})) \triangleq \varphi_{2V;\Theta}^{(L)}(\varphi_{1V;\Theta}^{(L)}(\mathbf{H}))$, where $\varphi_{2V;\Theta}^{(L)}(\cdot,\cdot)$ is the combination function to output the vertex representations in the Lth layer, $\varphi_{AV;\Theta}^{(L)}(\mathbf{H})$ denote the functions that map $\mathbf{H}$ to the aggregated output in the Lth layer (i.e., $[\mathbf{a}_{1,A}^{(L)}, \cdots, \mathbf{a}_{N,A}^{(L)}, \mathbf{a}_{1,U}^{(L)}, \cdots, \mathbf{a}_{K,U}^{(L)}]$), and $\varphi_{1V;\Theta}^{(L)}(\mathbf{H})$ denotes the functions that map $\mathbf{H}$ to $[\mathbf{d}_A^{(L-1)}, \mathbf{d}_U^{(L-1)}]$ and $[\mathbf{a}_{1,A}^{(L)}, \cdots, \mathbf{a}_{N,A}^{(L)}, \mathbf{a}_{1,U}^{(L)}, \cdots, \mathbf{a}_{K,U}^{(L)}]$. In order for $\varphi_{RV;\Theta}^{(L)}(\mathbf{H})$ not compressing channel information, $\varphi_{1V;\Theta}^{(L-1)}(\mathbf{H})$ (and hence $\varphi_{RV;\Theta}^{(L-1)}(\mathbf{H})$ and $\varphi_{AV;\Theta}^{(L)}(\mathbf{H})$) should not compress channel information. Hence, the output dimensions of $\varphi_{RV;\Theta}^{(L-1)}(\mathbf{H})$ and $\varphi_{AV;\Theta}^{(L)}(\mathbf{H})$ should not be less than the dimension of $\mathbf{H}$, i.e., $NM_A^{(L-1)} + KM_U^{(L-1)} \geq 2NK$ and $NM_{A,q}^{(L)} + KM_{U,q}^{(L)} \geq 2NK$. With similar analysis, we can show that the following conditions should be satisfied for the GNN not losing channel information,

$$\begin{aligned} NM_A^{(l)} + KM_U^{(l)} &\geq 2NK, \\ NM_{A,q}^{(l)} + KM_{U,q}^{(l)} &\geq 2NK, \\ l &= 1, 2, \cdots, L. \end{aligned} \tag{23}$$

When one sets $M_A^{(l)} = M_U^{(l)} \triangleq M_d^{(l)}$ and $M_{A,q}^{(l)} = M_{U,q}^{(l)} \triangleq M_q^{(l)}$, then the necessary conditions in (23) can be simplified as $M_d^{(l)} \geq 2NK/(N+K)$, $M_q^{(l)} \geq 2NK/(N+K)$, $l = 1, 2, \cdots, L$. Further considering the analysis in Section A, the Vertex-GNNs with linear processors cannot differentiate $\mathbf{H}_{[1]}$ and $\mathbf{H}_{[2]}$. Hence, to design a Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$ that does not lose channel information, the processing functions should be non-linear and the output dimensions of the processing and combination functions in (23) should be satisfied.

2) Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$

The structure of the GNN is shown in Fig. 4(b), where $\mathbf{d}_E^{(l)} \triangleq [\mathbf{d}_{11,E}^{(l)}, \cdots, \mathbf{d}_{NK,E}^{(l)}] \in \mathbb{R}^{M_E^{(l)}\times NK}$ is the combination

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

output of all edges, and $\mathbf{a}_{ij,E}^{(l)} \in \mathbb{R}^{M_{E,q}^{(l)}}$ is the aggregation output of edge (i,j) in the lth layer, and $M_{E,q}^{(l)}$ is a hyper-parameter.

The GNN can be expressed as composite functions $\varphi_{2E;\Theta}^{(L)}(\varphi_{RE;\Theta}^{(L-1)}(\mathbf{H}), \varphi_{AE;\Theta}^{(L)}(\mathbf{H})) \triangleq \varphi_{2E;\Theta}^{(L)}(\varphi_{1E;\Theta}^{(L)}(\mathbf{H}))$, where $\varphi_{2E;\Theta}^{(L)}(\cdot,\cdot)$ is the combination function to output the edge representations in the Lth layer (i.e., $\hat{\mathbf{V}}_E$), $\varphi_{RE;\Theta}^{(L-1)}(\mathbf{H})$ denote the functions that map $\mathbf{H}$ to the edge representations in the $(L-1)$th layer (i.e., $\mathbf{d}_E^{(L-1)}$), $\varphi_{AE;\Theta}^{(L)}(\mathbf{H})$ denote the functions that map $\mathbf{H}$ to the aggregated output in the Lth layer (i.e., $[\mathbf{a}_{11,A}^{(L)}, \cdots, \mathbf{a}_{NK,A}^{(L)}]$), and $\varphi_{1E;\Theta}^{(L)}(\mathbf{H})$ denote the functions that map $\mathbf{H}$ to $\mathbf{d}_E^{(L-1)}$ and $[\mathbf{a}_{11,A}^{(L)}, \cdots, \mathbf{a}_{NK,A}^{(L)}]$. If the GNN does not lose channel information, then $\varphi_{1E;\Theta}^{(L)}(\mathbf{H})$ (and hence $\varphi_{RE;\Theta}^{(L-1)}(\mathbf{H})$ and $\varphi_{AE;\Theta}^{(L)}(\mathbf{H})$) should not compress channel information. Hence, the output dimensions of $\varphi_{RE;\Theta}^{(L-1)}(\mathbf{H})$ and $\varphi_{AE;\Theta}^{(L)}(\mathbf{H})$ should not be less than the dimension of $\mathbf{H}$, i.e., $M_E^{(L-1)} \geq 2$ and $M_{E,q}^{(L)} \geq 2$. Analogously, we can show that the following conditions should be satisfied for the GNN not losing channel information,

$$\begin{aligned} &M_E^{(l)} \geq 2,\ l = 1, \cdots, L-1; \\ &M_{E,q}^{(l)} \geq 2,\ l = 1, \cdots, L. \end{aligned} \tag{24}$$

The dimensions should be at least 2, because $\mathbf{H}$ and $\mathbf{V}$ are complex matrices. Since $M_E^{(L)}$ is the dimension of the action vector defined on each edge, we have $M_E^{(L)} = 2$. Further recalling the analysis in Section A, to design an Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$ that does not compress channel information, the combination functions should be non-linear and the conditions in (24) should be satisfied.

V. SIMULATION RESULTS

In this section, we validate the previous analyses, and compare the system performance, space, time, and sample complexity of Vertex-GNNs with Edge-GNNs via simulations.

We consider three optimization problems, i.e, the link scheduling problem in (1), the power control problem in (2), and the precoding problem in (3). For the link scheduling and power control problems, all the D2D pairs are randomly located in a $500\ m \times 500\ m$ squared area. The wireless network parameters are provided in Table 5. The composite channel consists of the path loss generated with the model in [5], log-normal shadowing with standard deviation of 8 dB, and Rayleigh fading. For the precoding problem, we set $P_{max} = 1\ W$, and change σ_0^2 in (3) to adjust SNR. These simulation setups are considered unless otherwise specified.

While the GNNs can be trained in a supervised or unsupervised manner, we train the GNNs in an unsupervised manner to avoid generating labels that is time-consuming. Then, each sample only contains a channel matrix $\boldsymbol{\alpha} = [\alpha_{ij}]_{K\times K}$ that is generated according to the channel model with randomly located D2D pairs or $\mathbf{H} = [h_{nk}]_{N\times K}$ where each element follows Rayleigh distribution. We generate 5×10^5 samples

TABLE 5. Parameters in Simulation

Parameters	Values
D2D distance	2-65 m
Number of D2D links	50
Noise spectral density	-169 dBm/Hz
Bandwidth, Carrier frequency	5 MHz, 2.4 GHz
Antenna height, Antenna gain	1.5 m, 2.5 dBi
Transmit power of activation link	40 dBm

as the training set (the number of samples used for training may be much smaller), and 10^3 samples as the test set. Adam is used as the optimizer. The loss function is designed as $Loss = -\frac{1}{N_s}\sum_{n=1}^{N_s}\left(\sum_{k=1}^{K} r_k^n + w_1 \sum_{k=1}^{K}\log(y_k^n) + w_2\sum_{k=1}^{K}\log(1-y_k^n)\right)$, where N_s is the number of training samples, r_k^n is the data rate of the kth user and y_k^n is the activate probability of the kth D2D link in the nth sample, w_1 and w_2 are weights that need to be tuned. The second and the third terms in the loss function are respectively the penalty for preventing all the links from being closed and being activated. For the link scheduling problem, we set $w_1 = 10^{-1}$ and $w_2 = 10^{-4}$. For the power control and precoding problems, $w_1 = w_2 = 0$.

We use sum rate ratio as the performance metric. It is the ratio of the sum rate achieved by the learned policy to the sum rate achieved by a numerical algorithm (which is FPLinQ [2] for link scheduling, and WMMSE for power control and precoding). We train each GNN five times. The results are obtained by averaging the sum rate ratios achieved by the learned policies with the five trained GNNs over all test samples. For the link scheduling and power control problems, we only provide the performance of Vertex- and Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ in the sequel since Vertex- and Edge-GNN$_{\mathsf{ls}}^{\mathsf{undir}}$ perform very close to them.

All the simulation results are obtained on a computer with a 28-core Intel i9-10904X CPU, a Nvidia RTX 3080Ti GPU, and 64 GB memory.

All the GNNs in the following use the mean-pooling function.

A. IMPACT OF THE NON-DISTINGUISHABLE CHANNEL MATRICES

We first validate that the vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ cannot well learn the link scheduling and power control policies, and the vanilla Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$ cannot well learn the precoding policy, due to their weak expressive power.

In Fig. 5(a) and Fig. 5(b), we show the probability of $F_{\mathsf{ls}}(\boldsymbol{\alpha}_{[1]}) = F_{\mathsf{ls}}(\boldsymbol{\alpha}_{[2]})$ and $F_{\mathsf{pc}}(\boldsymbol{\alpha}_{[1]}) = F_{\mathsf{pc}}(\boldsymbol{\alpha}_{[2]})$ simulated with different values of K and transmit power, where $p_k = P$ for the link scheduling problem and $P_{\max} = P$ for the power control problem. $\boldsymbol{\alpha}_{[1]}$ and $\boldsymbol{\alpha}_{[2]}$ are generated by solving the linear equations in (13). Since the computational complexity of solving (13) is high when K is large, we take $K \in \{3,4,5,6\}$ as examples. For the link scheduling

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

problem, the optimal solutions are obtained by exhaustive searching. For the power control problem, the sub-optimal solutions are obtained by the WMMSE algorithm. Since the optimized powers are continuous, we regard the frequency of $|F_{\mathsf{pc}}(\boldsymbol{\alpha}_{[1]}) - F_{\mathsf{pc}}(\boldsymbol{\alpha}_{[2]})|_{max} < 10^{-3}$ as the probability. We can see that the probability decreases with K and P. This can be explained as follows. Since the solution spaces of both problems become large with a large value of K, the probability that the optimal solutions for $\boldsymbol{\alpha}_{[1]}$ and $\boldsymbol{\alpha}_{[2]}$ are identical decreases with K. When P is low such that the noise power dominates, the optimization for the K D2D pairs in problem (1) and (2) is decoupled. In this case, the optimal scheduling policy is to activate all the links, and the optimal power control policy is to transmit with the maximal power to all the receivers. Hence, given any two channel gain matrices, the two optimal solutions are always identical.

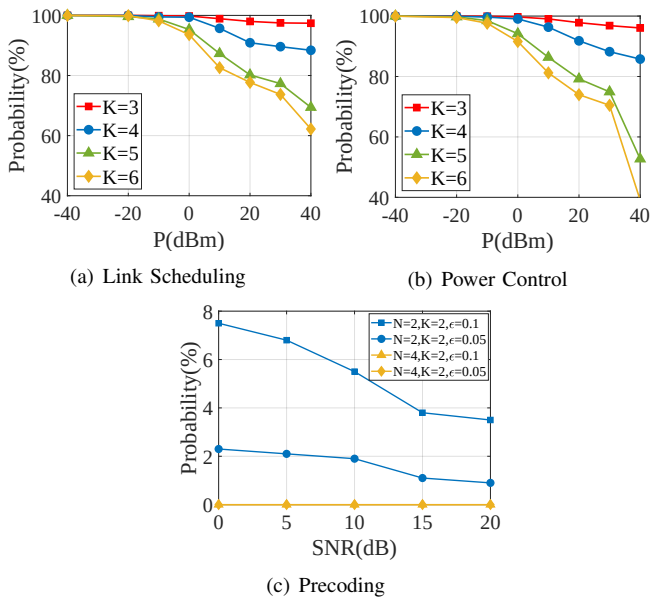


FIGURE 5. **Probability of (a) $F_{\mathsf{ls}}(\boldsymbol{\alpha}_{[1]}) = F_{\mathsf{ls}}(\boldsymbol{\alpha}_{[2]})$, (b) $F_{\mathsf{pc}}(\boldsymbol{\alpha}_{[1]}) = F_{\mathsf{pc}}(\boldsymbol{\alpha}_{[2]})$, and (c) $F_{\mathsf{p}}(\mathbf{H}_{[1]}) = F_{\mathsf{p}}(\mathbf{H}_{[2]})$, where $\boldsymbol{\alpha}_{[1]}$ and $\boldsymbol{\alpha}_{[2]}$ satisfy the conditions in Observation 1, and $\mathbf{H}_{[1]}$ and $\mathbf{H}_{[2]}$ satisfy the conditions in Section 2.**

In Fig. 5(c), we show the probability of $F_{\mathsf{p}}(\mathbf{H}_{[1]}) = F_{\mathsf{p}}(\mathbf{H}_{[2]})$ simulated with $N = 2$ or 4 and $K = 2$, where $\mathbf{H}_{[1]}$ and $\mathbf{H}_{[2]}$ are generated by solving the linear equations in (14). The sub-optimal solutions are also obtained by the WMMSE algorithm. Since the precoding variables are continuous, we regard the frequency of $|F_{\mathsf{p}}(\mathbf{H}_{[1]}) - F_{\mathsf{p}}(\mathbf{H}_{[2]})|_{max} < \epsilon$ as the probability, where ϵ is a parameter. We can see that the probability is low under different values of ϵ and SNR. When ϵ is smaller than 0.05, or the values of N and K are larger, the probability is almost zero, i.e., the optimized precoding matrices for $\mathbf{H}_{[1]}$ and $\mathbf{H}_{[2]}$ are different with very high probability.

In Fig. 6(a) and Fig. 6(b), we show the performance of the link scheduling and power control policies learned by the vanilla GNNs versus P. The GNNs are trained with 1000 samples, and the fine-tuned hyper-parameters are shown in Table 6. We can see that the performance of the vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ degrades rapidly with P, while the performance of the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ changes little with P. This is because when the value of P is higher, the probability of $F_{\mathsf{ls}}(\boldsymbol{\alpha}_{[1]}) = F_{\mathsf{ls}}(\boldsymbol{\alpha}_{[2]})$ or $F_{\mathsf{pc}}(\boldsymbol{\alpha}_{[1]}) = F_{\mathsf{pc}}(\boldsymbol{\alpha}_{[2]})$ is lower according to Fig. 5, but the vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ yields the same output with the two channel matrices $\boldsymbol{\alpha}_{[1]}$ and $\boldsymbol{\alpha}_{[2]}$. Recall that the vanilla Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ is with the mean-pooling function. If max-pooling is used as in [6], the Vertex-GNN will perform much better when learning the link scheduling or power control policy, which however is still inferior to the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$. It is noteworthy that the sum rate ratio achieved by the link scheduling policy learned by the vanilla Edge-GNN exceeds 100% as shown in Fig. 6(a). This is because the sum rate ratio is the ratio of the sum rate achieved by the learned policy to the sum rate achieved by the FPLinQ algorithm that can only learn suboptimal solutions.

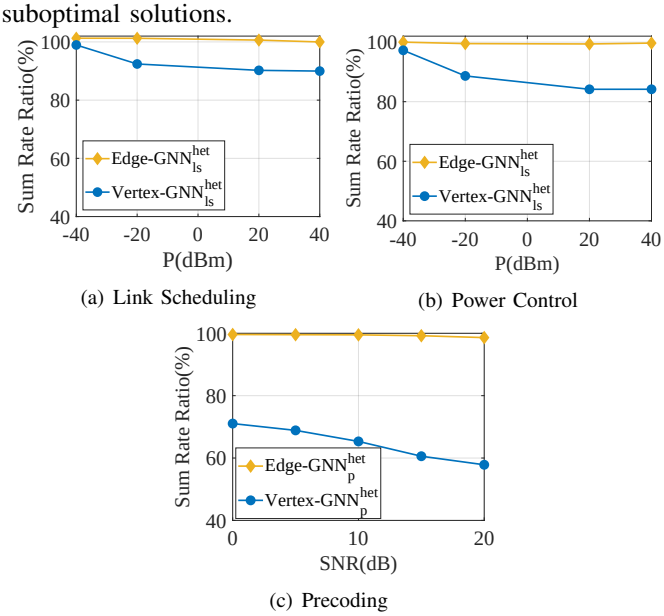


FIGURE 6. **System performance of the vanilla GNNs versus P or SNR. (a) and (b): $K = 50$. (c): $N = 4$, $K = 2$.**

In Fig. 6(c), we show the performance of the precoding policies learned by the vanilla GNNs versus SNR. The GNNs are trained with 10^5 samples, and the fine-tuned hyper-parameters are shown in Table 6. We can see that the vanilla Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$ performs poor under different SNRs. This is because the probability of $F_{\mathsf{p}}(\mathbf{H}_{[1]}) = F_{\mathsf{p}}(\mathbf{H}_{[2]})$ is low under different SNRs as shown in Fig. 5(c), but the vanilla Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$ yields the same output for $\mathbf{H}_{[1]}$ and $\mathbf{H}_{[2]}$. As expected, the vanilla Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$ performs very well.

B. Impact of the Linearity of Processing and Combination Functions

To validate the analysis in Section A, we compare the performance of Vertex-GNNs and Edge-GNNs with vanilla Vertex-GNNs and vanilla Edge-GNNs.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

TABLE 6. Fine-tuned hyper-parameters for the vanilla GNNs

Tasks	Parameters	Values	
		Vertex-GNN	Edge-GNN
Link Scheduling	Dimension of representation vectors in each layer	8,8,8,8,8,1	8,8,8,8,8,1
	Learning rate	0.001	0.01
Power control	Dimension of representation vectors in each layer	8,8,8,8,8,1	16,16,16,16,16,1
	Learning rate	0.001	0.01
Precoding	Dimension of representation vectors in each layer	32,32,32,32,32	32,32,32,32,32,2
	Number of neurons in the read-out layer	256,2	-[a]
	Learning rate	0.001	0.01

[a]In the vanilla Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$, there is no read-out layer.

The hyper-parameters for the vanilla GNNs are the same as Table 6. The fine-tuned hyper-parameters of the Vertex-GNNs with FNN as processor or combiner are shown in Table 7. The GNNs for learning the link scheduling policy and the power control policy are trained with 1000 samples. The GNNs for learning the precoding policy with $N = 4$, $K = 2$, and $\mathsf{SNR} = 10$ dB are trained with 10^5 samples. In Table 8, we provide the simulation results, where the performance of the vanilla GNNs is marked with bold font.

TABLE 7. Fine-tuned hyper-parameters for the Vertex-GNNs with FNN as processor or combiner

Tasks	Parameters	Values
Link Scheduling	Dimension of representation vectors in each layer	8,8,8,8,8,1
	Number of neurons in each hidden layer of FNN	32
	Learning rate	0.01
Power control	Dimension of representation vectors in each layer	16,16,16,16,16,1
	Number of neurons in each hidden layer of FNN	32
	Learning rate	0.001
Precoding	Dimension of representation vectors in each layer	16,16,16
	Number of neurons in each hidden layer of FNN	256
	Number of neurons in the read-out layer	256,2
	Learning rate	0.001

It is shown that for the Vertex-GNNs learning the same policy, when the processing function is FNN (no matter if the combiner is linear or non-linear, where the results for linear combiner are not shown because combiner is usually non-linear), the performance is much better than the vanilla Vertex-GNN. When only the combination function is FNN, the Vertex-GNN performs closely to the vanilla Vertex-GNN, because both of them cannot differentiate $\boldsymbol{\alpha}_{[1]}$ and $\boldsymbol{\alpha}_{[2]}$ or $\mathbf{H}_{[1]}$ and $\mathbf{H}_{[2]}$.

For the Edge-GNNs learning the link scheduling and power control policies, when $\mathsf{CB}_S(\cdot)$ is non-linear and $\mathsf{CB}_I(\cdot)$ is linear, Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ is inferior to the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$. When $\mathsf{CB}_I(\cdot)$ is non-linear and $\mathsf{CB}_S(\cdot)$ is linear, Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ performs closely to the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$, since both of them can differentiate $\boldsymbol{\alpha}_{[1]}$ and $\boldsymbol{\alpha}_{[2]}$. When learning the precoding policy, the Edge-GNN with linear combination function is inferior to the vanilla Edge-GNN, since it cannot differentiate $\mathbf{H}_{[1]}$ and $\mathbf{H}_{[2]}$.

Next, we show the impact of different activation functions. For the Vertex-GNNs, when the processing function is a non-linear function such as FNN, they can perform well even if a linear combination function is applied. Hence, non-linear activation functions in the combiner have little impact on the performance of the Vertex-GNNs. For the vanilla GNNs, only the combination functions contain non-linear activation functions. Thereby, we only compare the performance of the vanilla GNNs with several non-linear activation functions. The simulation results are provided in Table 9. It shows that the vanilla GNNs with different activation functions achieve similar performance.

C. Impact of the Output Dimensions of Processing and Combination Functions

To validate the analysis in Section B. we provide the performance of the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$, the vanilla Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$, as well as Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ and Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$ with FNN as processor (denoted as Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$+FNN and Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN, respectively).[3]

The hyper-parameters of the GNNs for learning the link scheduling and power control policies are provided in Tables 6 and 7. The hyper-parameters of the GNNs for learning the precoding policy with $N = 4$ and $K = 2$ are also shown in Tables 6 and 7. When using the GNNs to learn the precoding policy with $N = 8$ and $K = 4$, the number of training samples needs to increase to achieve acceptable performance, which is set to be 5×10^5. Their hyper-parameters are as follows. For the Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN, $L = 5$, the processing

[3]The features and actions of Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$+FNN and Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN are respectively the same as those of Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ and Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$ in Table. 3.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

TABLE 8. Performance of GNNs with different processing and combination functions

GNNs						Performance		
	Processing		Combination			Link Scheduling	Power Control	Precoding
	Linear	FNN	Linear	$\sigma(\cdot)$	FNN			
Vertex-GNNs	✓	✗	✗	✓	✗	**89.98%**	**84.17%**	**64.94%**
	✓	✗	✗	✗	✓	90.06%(+0.09%)	84.44%(+0.32%)	65.12%(+0.28%)
	✗	✓	✗	✓	✗	99.83%(+10.95%)	99.65%(+18.45%)	99.08%(+52.57%)
Edge-GNNs	✓	✗	✗	✓	✗	**99.99%**	**99.67%**	**99.16%**
	✓	✗	CB_I	CB_S	✗	89.91%(-10.08%)	84.13%(-15.59%)	-
	✓	✗	CB_S	CB_I	✗	99.99%(+0.00%)	99.22%(-0.45%)	-
	✓	✗	✓	✗	✗	-	-	62.23%(-37.24%)

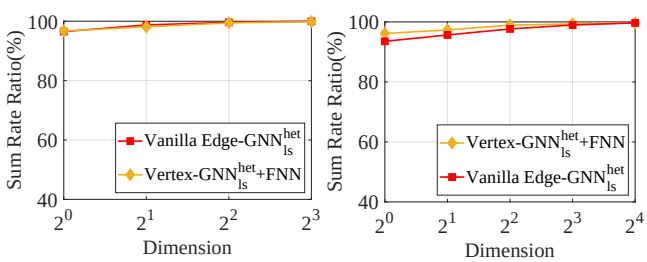
[1] $\sigma(\cdot)$ means that the combination function is a linear function cascaded with an activation function.

TABLE 9. Performance of vanilla GNNs under different non-linear activation functions

Tasks	GNNs	Activation functions					
		relu	leaky_relu	elu	swish	softplus	mish
		$\max(0,x)$	$\begin{cases} 0.2x & x<0 \\ x & x\geq 0 \end{cases}$	$\begin{cases} e^x-1 & x<0 \\ x & x\leq 0 \end{cases}$	$\frac{x}{1+e^{-x}}$	$ln(1+e^x)$	$x\cdot tanh(ln(1+e^x))$
Link Scheduling	Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$	89.98%	89.95%	90.08%	90.05%	90.21%	90.10%
	Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$	99.99%	99.99%	99.98%	99.83%	99.66%	100.18%
Power Control	Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$	84.18%	84.18%	84.36%	84.41%	84.30%	84.34%
	Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$	99.67%	98.39%	99.59%	99.49%	99.50%	99.61%
Precoding	Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$	65.17%	65.12%	65.14%	65.05%	65.17%	65.09%
	Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$	99.16%	99.14%	99.43%	99.49%	99.46%	99.58%

function is a FNN with a hidden layer with 256 neurons, the read-out layer is a FNN with five hidden layers each with 512 neurons. For the vanilla Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$, $L=9$.

In Fig. 7, we show the performance of the GNNs with different values of "Dimension" for learning the link scheduling and power control policies, where the "Dimension" indicates the value of $M^{(l)}$, $\forall l$. It is shown that the performance of the GNNs grows slowly with "Dimension". This is because the dimensions of the representation vectors do not affect the expressive power of the GNNs for learning these two policies, but the representation vectors with higher dimension lead to a wider GNN and hence provide larger hypothesis space.



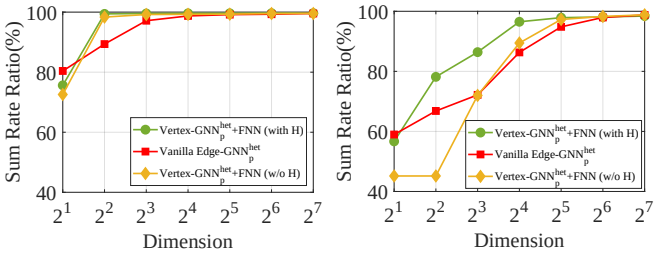
(a) Link Scheduling (b) Power Control

FIGURE 7. Performance of the GNNs with different dimensions of hidden representations, $K=50$.

In Fig. 8, we show the performance of the GNNs with different values of "Dimension" for learning the precoding policy where the "Dimension" indicates the value of $M_A^{(l)}=M_U^{(l)}=M_{A,q}^{(l)}=M_{U,q}^{(l)}=M_E^{(l)}=M_{E,q}^{(l)}$, $\forall l$. We show the impact of the output dimensions of the processors and combiners in this simple way, because there are too many combinations of these dimensions that can satisfy (23) or (24). Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN (w/o H) is a Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN, where only the representations of vertices are input to the read-out layer (i.e., $\hat{v}_{ij,V}=\mathsf{FNN}_{\mathsf{read}}^{\mathsf{het}}(\mathbf{d}_{i,A}^{(L)},\mathbf{d}_{j,U}^{(L)})$ as in Remark 3. Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN (with H) is another Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN, where the channel matrix $\mathbf{H}$ are also input to the read-out layer (i.e., $\hat{v}_{ij,V}=\mathsf{FNN}_{\mathsf{read}}^{\mathsf{het}}(\mathbf{d}_{i,A}^{(L)},\mathbf{d}_{j,U}^{(L)},h_{ij})$). It is shown that all the GNNs perform better with larger "Dimension". For Edge-GNN, (24) is satisfied when "Dimension"≥ 2. For Vertex-GNN, (23) is satisfied when "Dimension"≥ 3 for $N=4$ and $K=2$ and when "Dimension"≥ 6 for $N=8$ and $K=4$. However, even when (24) or (23) is satisfied, the vanilla Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$ or Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN (w/o H) may not perform well. This is because the conditions in (24) or (23) are only necessary for a GNN to avoid information loss. With the same "Dimension", the Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN (with H) performs better than the Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN (w/o H). This is because the input of the read-out layer of Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN (with H) contains $\mathbf{H}$, which is helpful to differentiate channel matrices.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872



(a) $N = 4,\ K = 2$ (b) $N = 8,\ K = 4$

FIGURE 8. Performance of the GNNs with different dimensions of hidden representations, precoding, SNR $= 10$ dB.

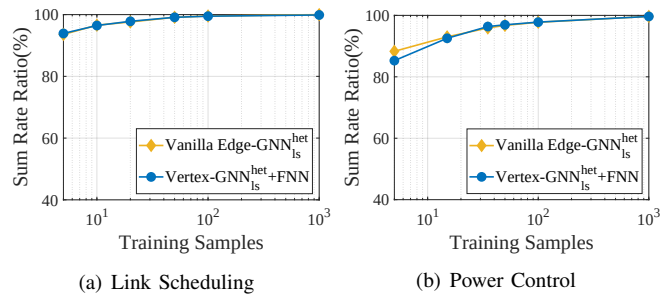
D. Space, Time, and Sample Complexities of the GNNs

Finally, we compare the space, time, and sample complexities of the Vertex-GNNs and the vanilla Edge-GNNs. The space complexity is the number of trainable parameters in a fine-tuned GNN to achieve an expected performance, which is set as 98% sum rate ratio for the precoding problem with $N = 8, K = 4$, and 99% for the other two problems and the precoding problem with $N = 4, K = 2$. The time complexity includes the training time required to achieve the expected performance and the inference time. The sample complexity is the minimal number of samples required for training a GNN to achieve a given performance.

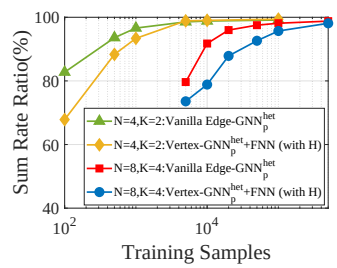
Since the Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN (with H) outperforms the Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN (w/o H), we only consider the Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN (with H). The Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$+FNN and the vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$ are with the hyper-parameters in Table 7 and Table 6, respectively. The Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN (with H) and Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$ are with the hyper-parameters in Section C. Besides, $M_d = 4$ for Vertex-GNN and $M_d = 32$ for Edge-GNN when $N = 4$ and $K = 2$, while $M_d = 64$ for Vertex-GNN and $M_d = 128$ for Edge-GNN when $N = 8$ and $K = 4$.

In Table 10, we show the space and time complexities of the GNNs. It is shown that the training time and inference time of the vanilla Edge-GNNs are shorter than the Vertex-GNNs, because using FNN as processor is with higher computational complexity than using linear processor. The space complexities of the vanilla Edge-GNNs are lower than the Vertex-GNNs.

In Fig. 9, we compare the performance of the GNNs trained with different numbers of samples. It is shown that the Edge-GNN and the Vertex-GNN for link scheduling are with almost the same sample complexity, while the Edge-GNN outperforms the Vertex-GNN with few number of training samples when learning the power control policy. When learning the precoding policy, the Edge-GNN (which uses linear processor) is more sample efficient than the Vertex-GNN, because the Vertex-GNN has more trainable parameters than the Edge-GNN due to using FNN as processor and the extra read-out layer.



(a) Link Scheduling (b) Power Control



(c) Precoding

FIGURE 9. Performance of the GNNs versus the number of training samples.

The main results for the three policies are summarized as follows.

- The vanilla Edge-GNNs outperform the vanilla Vertex-GNNs and perform closely to the Vertex-GNN with FNN-processor. The performance of the Vertex-GNNs can be improved by using FNN as processor, but cannot be improved by using FNN as combiner.
- The vanilla Edge-GNNs are with lower time complexity than the Vertex-GNN with FNN-processor to achieve

TABLE 10. Space and time complexity of GNNs

Tasks		GNN	Space (k)	Training Time (min)	Inference Time (ms)
Link Scheduling		Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$+FNN	12.74	4.43	20.58
		Vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$	2.14	2.08	5.16
Power Control		Vertex-GNN$_{\mathsf{ls}}^{\mathsf{het}}$+FNN	24.71	25.26	19.98
		Vanilla Edge-GNN$_{\mathsf{ls}}^{\mathsf{het}}$	8.37	5.62	5.36
Precoding	$N = 4, K = 2$	Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN (with H)	149.86	12.20	7.36
		Vanilla Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$	126.72	10.95	4.04
	$N = 8, K = 4$	Vertex-GNN$_{\mathsf{p}}^{\mathsf{het}}$+FNN (with H)	1454.34	717.82	10.87
		Vanilla Edge-GNN$_{\mathsf{p}}^{\mathsf{het}}$	345.60	23.13	4.92

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

the same performance. When learning the precoding policy, the vanilla Edge-GNN is with lower sample complexity than the Vertex-GNN with FNN-processor.

VI. Conclusion

In this paper, we analyzed the impacts of the linearity and output dimensions of processing and combination functions on the expressive power of the Vertex-GNNs and Edge-GNNs for learning link scheduling, power control, and precoding policies. We demonstrated that all the policies can be learned with either Vertex-GNNs or Edge-GNNs over either homogeneous or heterogeneous graphs. We showed that the Vertex-GNNs with linear processing functions cannot perform well for these policies due to their inability to differentiate all channel matrices. When learning the precoding policy, the expressive power of the Vertex-GNN with non-linear processing functions is still weak, which depends on the output dimensions of processing and combination functions. Simulation results showed that the Edge-GNNs using linear processors can achieve the same performance as the Vertex-GNNs using non-linear processors for learning these policies but with much lower training time and inference time. Our results indicate the advantage of Edge-GNNs for learning wireless policies and provide guidelines for designing efficient and well-performed GNNs. While we focused on two resource allocation policies and a precoding policy, the conclusions are also applicable to other wireless policies such as signal detection, channel estimation, other resource allocation, and other precoding problems whenever the edges of constructed graphs are with features. If both the features and actions of a graph are defined on vertices, then a Vertex-GNN will be with the same expressive power as an Edge-GNN and may be more sample efficient.

REFERENCES

[1] Qingjiang Shi, Meisam Razaviyayn, Zhiquan Luo, and Chen He. An iteratively weighted MMSE approach to distributed sum-utility maximization for a MIMO interfering broadcast channel. *IEEE Trans. Signal Process.*, 59(9):4331–4340, Sept. 2011.

[2] Kaiming Shen and Wei Yu. FPLinQ: A cooperative spectrum sharing strategy for device-to-device communications. *IEEE ISIT*, 2017.

[3] Haoran Sun, Xiangyi Chen, Qingjiang Shi, Mingyi Hong, Xiao Fu, and Nikos D. Sidiropoulos. Learning to optimize: Training deep neural networks for wireless resource management. *IEEE SPAWC*, 2017.

[4] Mark Eisen and Alejandro Ribeiro. Optimal wireless resource allocation with random edge graph neural networks. *IEEE Trans. Signal Process.*, 68:2977–2991, 2020.

[5] Mengyuan Lee, Guanding Yu, and Geoffrey Ye Li. Graph embedding-based wireless link scheduling with few training samples. *IEEE Trans. Wireless Commun.*, 20(4):2282–2294, Apr. 2021.

[6] Yifei Shen, Yuanming Shi, Jun Zhang, and Khaled B. Letaief. Graph neural networks for scalable radio resource management: Architecture design and theoretical analysis. *IEEE J. Sel. Areas Commun.*, 39(1):101–115, Jan. 2021.

[7] Shiwen He, Shaowen Xiong, Yeyu Ou, Jian Zhang, Jiaheng Wang, Yongming Huang, and Yaoxue Zhang. An overview on the application of graph neural networks in wireless networks. *IEEE Open J. Commun. Soc.*, 2:2547–2565, 2021.

[8] Mengyuan Lee, Guanding Yu, Huaiyu Dai, and Geoffrey Li. Graph neural networks meet wireless communications: Motivation, applications, and future directions. *IEEE Wireless Commun.*, 29(5):12–19, Oct. 2022.

[9] Jia Guo and Chenyang Yang. Learning power allocation for multi-cell-multi-user systems with heterogeneous graph neural networks. *IEEE Trans. Wireless Commun.*, 21(2):884–897, Feb. 2022.

[10] Xu Keyulu, Hu Weihua, Leskovec Jure, and Jegelka Stefanie. How powerful are graph neural networks? *ICLR*, 2019.

[11] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Trans. Neural Netw. Learn Syst.*, 32(1):4–24, Jan. 2021.

[12] Xiaodong Jiang, Pengsheng Ji, and Sheng Li. CensNet: Convolution with edge-node switching in graph neural networks. *IJCAI*, 2019.

[13] Jaehyeong Jo, Jinheon Baek, Seul Lee, Dongki Kim, Minki Kang, and Sungju Hwang. Edge representation learning with hypergraphs. *NeurIPS*, 2021.

[14] Xikun Zhang, Chang Xu, Xinmei Tian, and Dacheng Tao. Graph edge convolutional neural networks for skeleton-based action recognition. *IEEE Trans. Neural Netw. Learn. Syst.*, 31(8):3047–3060, Aug. 2020.

[15] Tianrui Chen, Xinruo Zhang, Minglei You, Gan Zheng, and Sangarapillai Lambotharan. A GNN-based supervised learning framework for resource allocation in wireless IoT networks. *IEEE Internet Things J.*, 9(3):1712–1724, Feb. 2022.

[16] Shiwen He, Shaowen Xiong, Wei Zhang, Yiting Yang, Ju Ren, and Yongming Huang. GBLinks: GNN-based beam selection and link activation for ultra-dense D2D mmWave networks. *IEEE Trans. Commun.*, 70(5):3451–3466, May. 2022.

[17] Xiaochen Zhang, Haitao Zhao, Jun Xiong, Xiaoran Liu, Li Zhou, and Jibo Wei. Scalable power control/beamforming in heterogeneous wireless networks with graph neural networks. *IEEE GLOBECOM*, 2021.

[18] Tao Jiang, Hei Victor Cheng, and Wei Yu. Learning to reflect and to beamform for intelligent reflecting surface with implicit channel estimation. *IEEE J. Sel. Areas Commun.*, 39(7):1931–1945, Jul. 2021.

[19] Baichuan Zhao, Jia Guo, and Chenyang Yang. Learning precoding policy: CNN or GNN? *IEEE WCNC*, 2022.

[20] Junbeom Kim, Hoon Lee, Seung-Eun Hong, and Seok-Hwan Park. A bipartite graph neural network approach for scalable beamforming optimization. *IEEE Trans. Wireless Commun.*, 22(1):333–347, Jan. 2023.

[21] Shengjie Liu, Jia Guo, and Chenyang Yang. Multidimensional graph neural networks for wireless communications. *IEEE Trans. Wireless Commun.*, early access,2023.

[22] Vismika Ranasinghe, Nandana Rajatheva, and Matti Latva-aho. Graph neural network based access point selection for cell-free massive MIMO systems. *IEEE GLOBECOM*, 2021.

[23] Chengjian Sun, Jiajun Wu, and Chenyang Yang. Improving Learning Efficiency for Wireless Resource Allocation with Symmetric Prior. *IEEE Wireless Commun.*, 29(2):162–168, Apr. 2022.

[24] Pan Li and Jure Leskovec. *Graph Neural Networks: Foundations, Frontiers, and Applications*, chapter The Expressive Power of Graph Neural Networks, pages 63–98. Springer Nature Singapore, Singapore, 2022.

[25] Sato Ryoma. A survey on the expressive power of graph neural networks. *arXiv:2003.04078*, 2020.

[26] Stefanie Jegelka. Theory of graph neural networks: Representation and learning. *arXiv:2204.07697*, 2022.

[27] Chuxu Zhang, Dongjin Song, Chao Huang, Ananthram Swami, and Nitesh V. Chawla. Heterogeneous graph neural network. *SIGKDD*, 2019.

[28] Jo Jaehyeong, Baek Jinheon, Lee Seul, Kim Dongki, Wang Minki, and Hwang. Sung, Ju. Edge representation learning with hypergraphs. *NeurIPS*, 2021.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Transactions on Machine Learning in Communications and Networking. This is the author's version which has not been fully edited an
content may change prior to final publication. Citation information: DOI 10.1109/TMLCN.2024.3354872

Yao Peng received her B.S. degree in electronics engineering from Beihang University, China, in 2021. She is currently pursuing her Ph.D. degree in signal and information processing with the School of Electronics and Information Engineering, Beihang University. Her research interests include graph neural network and its applications in wireless communications.

Jia Guo (Graduate Student Member, IEEE) received his B.S. degree in electronics engineering, and Ph.D. degree in information and communication engineering from Beihang University, China, in 2016 and 2023, respectively. He is currently a Postdoctoral Researcher with Beihang University. His research interests lie in the area of machine learning for wireless communications.

Chenyang Yang (Senior Member, IEEE) received her Ph.D. degree in Electrical Engineering from Beihang University, China, in 1997. She has been a full professor with Beihang University since 1999. She has published over 200 papers in the fields of machine learning for wireless communications, URLLC, energy efficient resource allocation, wireless caching, interference management, etc. She was supported by the 1st Teaching and Research Award Program for Outstanding Young Teachers of Higher Education Institutions by Ministry of Education of China. She has served as an associate or guest editor for several IEEE journals. Her recent research interests include mobile/wireless AI, and URLLC.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/